

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Programa de Licenciatura en Ingeniería en Electrónica



Implementación de un subsistema multiagente de estimación de pose para el rover Olympus

Informe de Trabajo Final de Graduación para optar por el título
de Ingeniero en Electrónica con el grado académico de Licenciatura

Jorge Schofield Carvajal

Profesor asesor:
Dr. Johan Carvajal Godínez

Cartago, Costa Rica
Mes, Día, 2026

Declaratoria de autenticidad

Declaratoria de uso de Inteligencia Artificial

Declaro que el presente documento de tesis ha sido realizado enteramente por mi persona, utilizando y aplicando literatura referente al tema introduciendo conocimientos, análisis y resultados experimentales propios. En los casos en que he utilizado bibliografía he procedido a indicar las fuentes mediante las respectivas citas bibliográficas. En consecuencia, asumo la responsabilidad total por el trabajo de tesis realizado y por el contenido del presente documento. Asimismo, en concordancia con la política de uso de herramientas de Inteligencia Artificial (IA) para los programas de posgrado del Tecnológico de Costa Rica, se declara de forma explícita el uso limitado y complementario de herramientas de IA generativa durante el desarrollo de esta tesis. Se emplearon modelos de lenguaje de gran escala (LLMs) y agentes de programación, tales como ChatGPT, Codex y herramientas similares, exclusivamente como apoyo para:

- Mejorar la claridad, estructura y redacción técnica del documento.
- Revisar aspectos de gramática, estilo y coherencia entre secciones.
- Generar y depurar scripts auxiliares utilizados durante el procesamiento de datos, entrenamiento y evaluación.
- Apoyar la verificación, revisión y depuración de fragmentos de código.
- Sugerir formas de organización de tablas, figuras y resultados.

El grado de dependencia de estas herramientas fue medio en tareas de redacción y organización del documento, y medio en tareas relacionadas con programación y verificación de código. Las ideas centrales de la investigación, la formulación del problema, el diseño experimental, la arquitectura de los modelos, la interpretación de resultados y las conclusiones corresponden exclusivamente al autor. La inteligencia artificial no fue utilizada para generar el contenido intelectual sustantivo de la tesis ni se considera autora o coautora del presente trabajo. Toda sugerencia, texto, fragmento de código o recomendación generada mediante herramientas de IA fue revisada críticamente, adaptada y validada antes de su incorporación al documento o al desarrollo experimental.

Esta declaración se incluye con el fin de garantizar la transparencia, la trazabilidad y el uso ético y responsable de herramientas de inteligencia artificial, de acuerdo con los principios de integridad académica, autoría y responsabilidad establecidos por el Tecnológico de Costa Rica.

Resumen

Abstract

Índice general

1. Introducción	8
1.1. Entorno del proyecto	8
1.2. Planteamiento del problema	10
1.2.1. Descripción del problema	10
1.2.2. Justificación	11
1.2.3. Síntesis del problema	11
1.3. Meta y objetivos	12
1.3.1. Meta	12
1.3.2. Objetivo general	12
1.3.3. Objetivos específicos	12
1.3.4. Entregables	13
1.4. Enfoque metodológico	13
1.4.1. Contingencia para la adaptación de CMAES	13
1.4.2. Cronograma	16
1.5. Enfoque de la solución	19
1.6. Alcances y limitaciones del proyecto	20
1.7. Recursos y presupuesto	21
2. Marco teórico	23
2.1. Fundamentos conceptuales	23
2.1.1. Autonomía y el problema de la localización	23
2.1.2. Restricciones de la exploración planetaria	24
2.1.3. Sistemas de referencia y representación de la pose	24
2.1.4. Cinemática del vehículo	26
2.1.5. Sensores y caracterización de sus errores	27
2.1.6. Estimación de estado	30
2.1.7. Arquitecturas de software para sistemas embebidos	32
2.1.8. Métodos de validación	33
2.2. Revisión de literatura	34
2.2.1. De la odometría pura a la fusión inercial	34
2.2.2. Cinemática de vehículos que giran por deslizamiento	35
2.2.3. Estimación de pose en róveres de exploración	35
2.2.4. Arquitecturas de software multiagente para sistemas embebidos	36

2.3.	Estado del arte	36
2.3.1.	Odometría visual e inercial y SLAM	36
2.3.2.	Otras formas de construir el estimador	36
2.3.3.	Tendencias en cómputo y software a bordo	37
2.4.	Síntesis y posicionamiento del subsistema	37
3.	Desarrollo del subsistema de estimación de pose	40
3.1.	Adaptación de la biblioteca CMAES a Linux	40
3.1.1.	Análisis de la biblioteca original	41
3.1.2.	Elección de la estrategia	42
3.1.3.	Diseño	43
3.1.4.	Implementación	46
3.1.5.	Pruebas	47
3.1.6.	Limitaciones	48
3.2.	Diseño del estimador	48
3.2.1.	Punto de partida	48
3.2.2.	Odometría de Olympus	49
3.2.3.	Elección del vector de estado	50
3.2.4.	Modelo de proceso	51
3.2.5.	Jacobiano de transición	52
3.2.6.	Ruido de proceso	52
3.2.7.	Modelo de medición	52
3.2.8.	Observabilidad del sesgo y actualización en reposo	53
3.2.9.	Detección de deslizamiento y adaptación	55
3.2.10.	Correspondencia con la implementación	55
3.2.11.	Resumen de decisiones de diseño	57
3.3.	Modelo de simulación de referencia	57
3.3.1.	De la primera versión a la versión 2.0	58
3.3.2.	Arquitectura	58
3.3.3.	Parámetros y su procedencia	60
3.3.4.	Verificación del modelo	60
3.3.5.	Resultados	61
3.3.6.	Hallazgos	61
3.3.7.	Diferencias entre el modelo y la implementación	68
3.3.8.	Consecuencias para el proyecto	68
3.4.	Diseño de los agentes del controlador de alto nivel	69
3.4.1.	Requerimientos	69
3.4.2.	Canal de sensores	69
3.4.3.	Descomposición en agentes	70
3.4.4.	Mensajes entre agentes	71
3.4.5.	Presupuesto de latencia	71
3.5.	Diseño preliminar de la interfaz eléctrica de la IMU	72
3.5.1.	Requerimientos	72
3.5.2.	Restricciones	72
3.5.3.	Alternativas	73
3.5.4.	Decisión	73

3.5.5. Entregables de hardware	74
4. Estado de avance del proyecto	75
4.1. Actividades realizadas	75
4.2. Reflexiones preliminares	76
4.3. Retos del proyecto	76
4.4. Siguietes pasos	79

Capítulo 1

Introducción

1.1. Entorno del proyecto

Este trabajo se realiza en el Laboratorio de Sistemas Espaciales (SETEC Lab) del Instituto Tecnológico de Costa Rica, bajo la asesoría del Dr. Johan Carvajal Godínez. La Escuela de Ingeniería Electrónica fundó el laboratorio en julio de 2017 para aprovechar el personal y el equipo que el TEC había desarrollado en el Proyecto Irazú, y es el primer laboratorio de su tipo en Centroamérica [1]. El Proyecto Irazú, desarrollado por el TEC y la Asociación Centroamericana de Aeronáutica y del Espacio, puso en órbita en mayo de 2018 un CubeSat de una unidad, el primer satélite de Costa Rica y de Centroamérica [2]. Después de validar tecnologías de potencia, comunicaciones y telemetría en misiones CubeSat, el laboratorio amplió su investigación hacia la movilidad planetaria autónoma [3].

En esa línea, el laboratorio desarrolla el proyecto ELANaV (*Extraterrestrial Landing and Navigation Vehicle*) que busca crear algoritmos de aterrizaje y navegación para vehículos de exploración en otros planetas. Hasta 2026, esos algoritmos solo se habían validado en simuladores como Gazebo y Webots [3]. Para probarlos en condiciones reales se necesita una plataforma física cuya arquitectura se parezca lo suficiente a la de un rover de exploración real, de modo que los resultados experimentales sean representativos.

Esa plataforma es el rover Olympus, desarrollado por Gómez Quesada [3]. Olympus tiene dos computadoras a cargo del procesamiento. La de bajo nivel (*Low level controller* o LLC) es un microcontrolador ATmega2560 con firmware en Rust, y la de alto nivel (*High level controller* o HLC) es una Raspberry Pi 5 con un sistema operativo basado en Linux construido con Yocto. El rover tiene además seis motores de corriente directa con encoders de efecto Hall, una suspensión (rocker-bogie) y un subsistema de percepción para navegación reactiva. La figura 1.1 muestra esta arquitectura con los elementos que agrega este trabajo, marcados en naranja.

Por otro lado, el SETEC Lab ha desarrollado una línea de investigación sobre

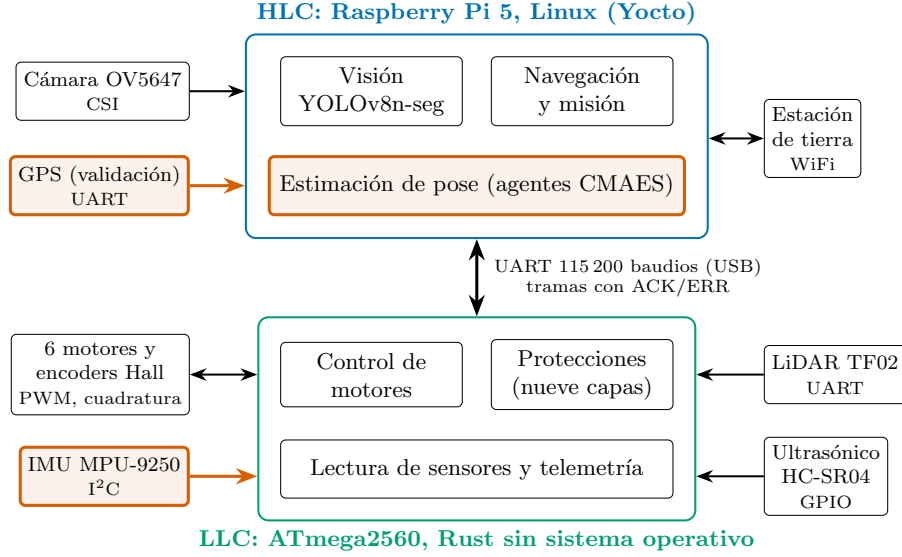


Figura 1.1: Arquitectura del rover Olympus con el subsistema de estimación de pose. Adaptado de [3].

software multiagente para sistemas embebidos. Esta línea parte de la investigación doctoral de Carvajal-Godínez en la Universidad Tecnológica de Delft, donde propuso una metodología para diseñar el software de a bordo de satélites pequeños como un conjunto de agentes, con el fin de hacerlo más tolerante a fallos [4]. A partir de esa metodología, Chan-Zheng desarrolló el marco de trabajo MAES para las computadoras de a bordo de satélites miniaturizados [5, 6]. Después, Rojas Marín llevó MAES al sistema operativo FreeRTOS con la biblioteca FreeMAES [7], y Conejo Boza escribió una versión en lenguaje C llamada CMAES [8]. De esta línea de trabajo salen el paradigma de diseño y la biblioteca que usa este proyecto.

Este trabajo une las dos líneas. Durante la campaña de verificación de Olympus se identificó un faltante concreto: el rover tiene el hardware de odometría y la interfaz para conectar una unidad de medición inercial (IMU), pero no tiene un algoritmo que combine esas mediciones para estimar su pose. El protocolo de telemetría ya incluye campos para la posición y la orientación, pero hoy se envían en cero, y el requisito de precisión de navegación quedó registrado como pendiente [3].

1.2. Planteamiento del problema

1.2.1. Descripción del problema

Un rover en otro planeta no tiene un sistema de navegación por satélite como el GPS. Además, las señales entre la Tierra y el rover tardan varios minutos en llegar, así que no es posible controlarlo en tiempo real desde la Tierra. Por eso, el rover debe calcular su posición y su orientación por sí mismo, con sus propios sensores.

Olympus no cuenta con un subsistema que estime su pose. Aunque la plataforma tiene los encoders de sus seis motores y una interfaz prevista para conectar una IMU, ningún algoritmo combina esas mediciones, y los campos de posición y orientación de su telemetría están desactivados [3]. Sin la pose, el rover no puede cerrar el ciclo de la navegación autónoma, que consiste en percibir el entorno, saber dónde está y decidir cómo moverse.

La forma más directa de estimar la pose, sumar el desplazamiento que miden los encoders, no es suficiente en Olympus por tres razones descritas en la literatura.

La primera viene de la forma en que gira el rover. Olympus cambia de dirección haciendo que las ruedas de un lado giren más rápido que las del otro, y en cada giro las llantas se arrastran de lado contra el suelo. En consecuencia, la orientación que se calcula con la diferencia de velocidad entre los dos lados tiene un error que se repite en cada giro [9]. Ese error en la orientación se traslada luego a la posición, y crece a medida que el rover avanza.

La segunda es que la odometría no puede distinguir entre una rueda que avanza y una rueda que patina. En terreno suelto, que es justamente el tipo de terreno donde se debe movilizar, el rover reporta que se movió aunque en realidad no lo hizo [10, 11].

La tercera es que el error de la odometría crece sin límite con la distancia recorrida porque cada pequeño error se suma al anterior, y depende de parámetros como el radio de las ruedas, las cuentas por vuelta de los encoders y el ancho de vía efectivo [10]. En la documentación de Olympus, esos parámetros solo tienen valores provisionales [3].

A esto se suma un problema de software. Estimar la pose implica varias tareas que ocurren al mismo tiempo: leer los sensores, filtrar las señales, ejecutar el algoritmo de estimación y comunicarse con otros componentes del rover. Si todas se programan en un solo procedimiento, el software se vuelve difícil de integrar y de ampliar. El paradigma de agentes, que el SETEC Lab ya usa en otras plataformas, resuelve esa organización [4], pero su biblioteca en C, CMAES, solo funciona sobre FreeRTOS [8], mientras que la computadora de alto nivel de Olympus usa Linux. La figura 1.2 resume estas causas y sus consecuencias.

A estos problemas se suma una restricción de arquitectura. Olympus ya está en uso, así que el nuevo subsistema debe integrarse sin quitarle capacidades. En la práctica, esto significa que no se puede cambiar por completo el formato de las tramas de telemetría ni las funciones que el software actual ya cumple. Los cambios al firmware del controlador de bajo nivel se limitan a leer la IMU y a

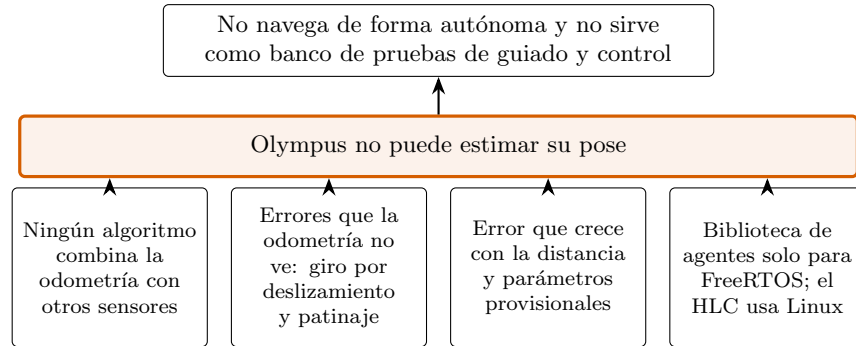


Figura 1.2: Causas y consecuencias de la falta de estimación de pose en Olympus. Basado en [3, 9, 10, 8].

corregir la forma en que mide el tiempo, sin cambiar lo que envía.

1.2.2. Justificación

Sin una estimación de pose, Olympus no puede cerrar el ciclo de la navegación autónoma: percibir el entorno, saber dónde está y decidir cómo moverse. Tampoco se puede verificar el requisito de precisión de navegación de ELANaV, que pide un error de odometría menor al 5 % y que quedó pendiente en la campaña de verificación de la plataforma [3]. La hoja de ruta de ELANaV hacia TRL-5 señala justamente la fusión de odometría e IMU como el siguiente paso [3].

Por otro lado, el trabajo continúa dos líneas de investigación propias del SETEC Lab, la plataforma Olympus y el software multiagente, y será la primera vez que la biblioteca CMAES funcione sobre Linux. La adaptación queda disponible para otros proyectos del laboratorio que usen computadoras con sistema operativo.

Finalmente, para probar algoritmos de navegación en Olympus, el personal del laboratorio debe operar el róver de forma manual, lo que consume tiempo y limita la cantidad y la variedad de los experimentos. Con una pose estimada a bordo, el laboratorio puede hacer más pruebas sin depender de operación manual ni de instrumentación externa. Además, las pruebas de campo de ELANaV se planean realizar en zonas volcánicas [3], y un róver que navega solo reduce la exposición del personal a gases, suelos inestables y temperaturas extremas.

1.2.3. Síntesis del problema

El róver Olympus no tiene un subsistema de estimación de pose que combine varias fuentes de medición. Por esto, no es posible validar sobre la plataforma los algoritmos de navegación del proyecto ELANaV, y se limita su avance hacia niveles de madurez tecnológica más altos.

1.3. Meta y objetivos

1.3.1. Meta

Contar con un subsistema funcional de estimación de pose, basado en la fusión de odometría por encoders e IMU bajo el paradigma de sistemas multiagente, integrado en el rover Olympus, que permita validar su desempeño mediante comparación con mediciones de referencia reales.

Indicador: El subsistema opera de forma continua sobre el rover Olympus, entrega estimaciones de pose a 10 Hz o más y mantiene un error de posición final no mayor al 3 % de la distancia recorrida en los tres escenarios de prueba del objetivo OE6, medido contra marcadores de referencia.

1.3.2. Objetivo general

Desarrollar un subsistema de estimación de pose para el rover Olympus utilizando el paradigma de agentes de software.

Indicador: Durante un recorrido UMBmark en interiores de 2 m por lado (8 m en total), ejecutado sobre Olympus con el subsistema completo, la pose se actualiza a 10 Hz o más y el error de posición final no supera el 3 % de la distancia recorrida (24 cm).

1.3.3. Objetivos específicos

OE1. Diseñar y caracterizar la interfaz eléctrica de los sensores nuevos: la IMU con el controlador de bajo nivel y el receptor GPS con el controlador de alto nivel.

Indicador: una medición con osciloscopio muestran que los niveles alto y bajo del bus I²C cumplen, en el ATmega2560 y en la IMU, los márgenes de ruido de la especificación I²C ($0,2 V_{DD}$ en el nivel alto y $0,1 V_{DD}$ en el nivel bajo) y que el tiempo de subida es menor que 1000 ns. Además, el LLC lee la IMU a 50 Hz durante 45 000 lecturas seguidas sin errores de comunicación.

OE2. Implementar la adquisición y la telemetría de los datos de los encoders, la IMU y el receptor GPS.

Indicador: el firmware del LLC envía al HLC, por el enlace UART, los datos de los seis encoders y de la IMU a 50 Hz o más, medidos con el reloj del HLC. El receptor GPS entrega coordenadas válidas al HLC con un refresco mínimo de 1 Hz.

OE3. Adaptar la biblioteca CMAES al sistema operativo Linux del controlador de alto nivel.

Indicador: la biblioteca compila en la Raspberry Pi 5 con la imagen de Yocto. Además, una versión del ejemplo de piedra, papel o tijera que envía mensajes a 50 Hz procesa al menos 45 000 mensajes, con registro, suspensión y reanudación de agentes, sin fallos y sin crecimiento de la memoria del proceso.

OE4. Diseñar la aplicación de estimación de pose basada en agentes sobre la biblioteca CMAES adaptada.

Indicador: la aplicación tiene al menos cuatro agentes funcionales (adquisición de datos, fusión de datos, estimación y comunicación) que intercambian mensajes por el bus de CMAES dentro del HLC. En una prueba de 15 min a 50 Hz, la tasa de entrega de mensajes es igual o superior al 99 % y no hay más de cuatro mensajes perdidos seguidos.

OE5. Integrar el sistema de estimación de pose, formado por el firmware del LLC y la aplicación de agentes del HLC, sobre la plataforma Olympus.

Indicador: el sistema integrado entrega estimaciones de pose con una latencia de extremo a extremo (desde la lectura en el LLC hasta la estimación en el HLC) inferior a 100 ms, de forma sostenida durante al menos 15 minutos continuos de operación sin fallos.

OE6. Evaluar la precisión del algoritmo de estimación de pose basado en agentes en comparación con el modelo de simulación y con pruebas físicas.

Indicador: en tres escenarios de prueba (una recta de 5 m, un giro en el lugar de 360° y un recorrido UMBmark de 2 m por lado), el error de la estimación, medido contra marcadores de referencia cumple estas cotas: el error de posición final no supera el 3 % de la distancia recorrida en la recta y en el recorrido UMBmark, y el error de rumbo final no supera el 3 % del giro total en el giro en el lugar (10,8°). La diferencia entre la estimación en el röver y la del modelo de simulación en los mismos escenarios cumple las mismas cotas.

1.3.4. Entregables

El cuadro 1.1 relaciona cada entregable con su objetivo, con el indicador que lo verifica y con la semana del cronograma (sección 1.4.2).

1.4. Enfoque metodológico

El proyecto se organiza en seis fases con un enfoque iterativo: cada componente se diseña, se implementa y se prueba por separado antes de integrarlo al sistema completo. Así, los errores se detectan temprano y no afectan el resto del desarrollo. Además, el diseño se guía por un modelo de simulación de la plataforma, que permite probar el estimador antes de llevarlo al röver y que sirve después como referencia para evaluarlo.

El cuadro 1.2 resume las actividades principales de cada fase, el resultado esperado y el objetivo específico con el que se relaciona.

1.4.1. Contingencia para la adaptación de CMAES

La adaptación de CMAES está en la ruta crítica. La aplicación de agentes, la integración y las pruebas dependen de ella. Si al final de la semana 8 la biblioteca no cumple el indicador del OE3 (hito H3), se activa esta contingencia. La base nueva se reemplaza por la opción que requiere menos trabajo de las evaluadas en la sección 3.1: compilar la versión de FreeRTOS sin cambios sobre el puerto oficial de FreeRTOS para POSIX. Esa opción no usa hilos reales de Linux, pero

Cuadro 1.1: Entregables del proyecto, objetivo relacionado, indicador que los verifica y semana de entrega.

Entregable	Objetivo	Indicador que lo verifica	Semana
Documento de requerimientos técnicos	OE1 a OE6	Aprobación del profesor asesor (hito H1)	3
Esquemático de la interfaz eléctrica de la IMU	OE1	Aprobación del profesor asesor (hito H2)	5
Diagrama de conexión de los sensores en Olympus	OE1	Aprobación del profesor asesor (hito H2)	5
Documento de arquitectura multiagente y protocolo serial LLC-HLC	OE2, OE4	Aprobación del profesor asesor (hito H2)	5
Modelo de referencia del róver en MATLAB y Simulink	OE6	Base de comparación del indicador del OE6	6
Informe de caracterización eléctrica medida	OE1	Indicador del OE1	6
Biblioteca CMAES para Linux con su informe de pruebas	OE3	Indicador del OE3 (hito H3)	8
Firmware del LLC con adquisición de encoders e IMU y módulo del GPS en el HLC	OE2	Indicador del OE2	8
Aplicación multiagente en el HLC	OE4	Indicador del OE4	10
Sistema integrado entre el LLC y el HLC	OE5	Indicador del OE5 (hito H4)	12
Informe de evaluación de precisión	OE6	Indicador del OE6 (hito H5)	14
Código fuente documentado y manual de integración	OE2 a OE5	Revisión del profesor asesor	16
Informe final y presentación de defensa	OE1 a OE6	Aprobación del tribunal	16

Cuadro 1.2: Fases del proyecto, actividades principales y resultados esperados.

Fase (semanas)	Actividades principales	Resultado esperado	Objetivo
1. Investigación y requerimientos (1–3)	Revisión del estado del arte en fusión de odometría e IMU y en sistemas multi-agente embebidos; resolución de los encoders; requerimientos cuantitativos de frecuencia, error, latencia e interfaz eléctrica	Documento de requerimientos técnicos	Todos
2. Diseño (4–6)	Arquitectura multi-agente sobre CMAES; interfaz eléctrica de la IMU y del GPS; ampliación del protocolo serial; selección del algoritmo de fusión; modelo de simulación del rover	Esquemático; arquitectura y protocolo documentados; modelo de referencia	OE1, OE4, OE6
3. Implementación (6–10)	Adaptación de CMAES a Linux; montaje y caracterización de la interfaz eléctrica; firmware del LLC; integración del GPS en el HLC; agentes; pruebas unitarias	Componentes probados por separado; informe de caracterización	OE1 a OE4
4. Integración (11–12)	Integración paso a paso en Olympus: LLC, agentes del HLC, comunicación con la navegación existente y GPS	Sistema integrado funcionando	OE5
5. Validación experimental (13–15)	Pruebas funcionales; comparación de la pose estimada con marcadores de referencia en los escenarios planteados; comparación con el modelo; ajustes de calibración	Informe de evaluación de precisión	OE6
6. Documentación y cierre (2–16)	Redacción continua desde la fase 1; informe final y defensa	Informe final	Todos

conserva la misma interfaz de agentes, así que la aplicación del OE4 se puede desarrollar sin cambios. El tiempo que tome el cambio se descuenta del margen de contingencia de la semana 15.

1.4.2. Cronograma

El proyecto se desarrolla en 16 semanas, del 3 de agosto al 20 de noviembre de 2026. La planificación tiene tres componentes: el diagrama de Gantt de la figura 1.3, la lista de actividades del cuadro 1.3 y los hitos de control del cuadro 1.4.

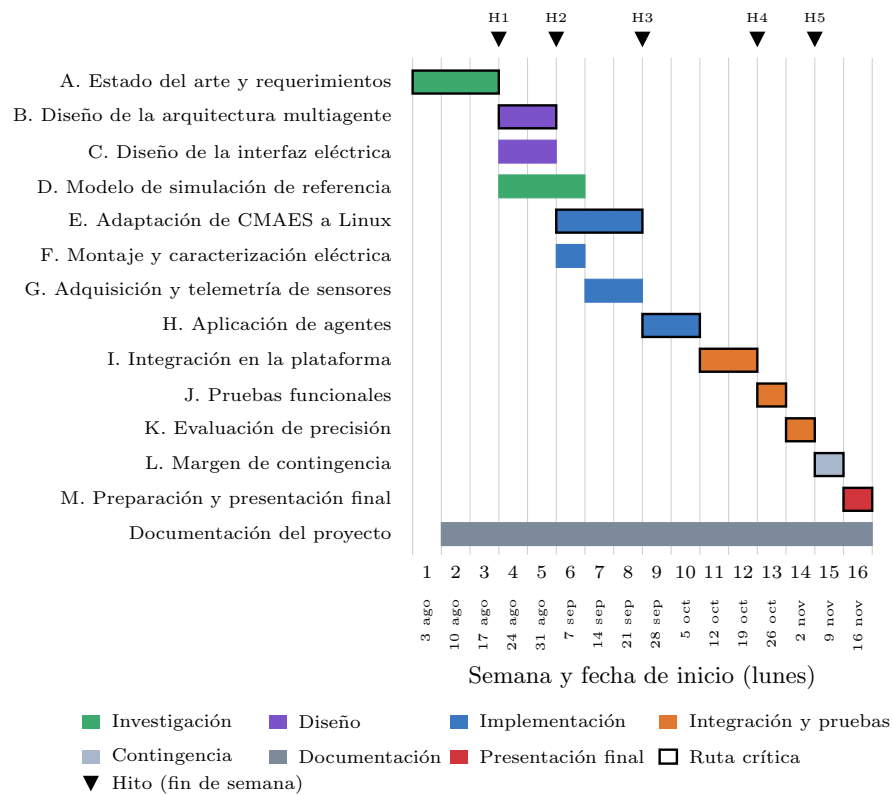


Figura 1.3: Cronograma del proyecto.

Actividades y ruta crítica

El cuadro 1.3 define cada actividad con su duración, sus actividades precedentes y sus semanas de inicio y fin más tempranas. Salvo dos excepciones, cada actividad comienza cuando terminan sus precedentes.

La ruta crítica es descrita por la secuencia A, B, E, H, I, J, K, L y M, cuyas actividades están resaltadas en la Figura 1.3. Incluye la adaptación de CMAES (E), de la que dependen la aplicación de agentes y todas las etapas siguientes. Las actividades fuera de la ruta crítica (el diseño de la interfaz eléctrica, el modelo de simulación, el montaje y la caracterización eléctrica, y la adquisición de datos de los sensores) tienen dos semanas de holgura. La documentación corre en paralelo de la semana 2 a la 16 y no tiene precedentes.

Cuadro 1.3: Red de actividades del proyecto. La duración y la holgura se expresan en semanas; el inicio y el fin son las semanas más tempranas posibles.

Id	Actividad	Duración	Precedentes	Inicio	Fin	Holgura
A	Estado del arte y requerimientos	3	–	S1	S3	0
B	Diseño de la arquitectura multiagente	2	A	S4	S5	0
C	Diseño de la interfaz eléctrica	2	A	S4	S5	2
D	Modelo de simulación de referencia	3	A	S4	S6	2
E	Adaptación de CMAES a Linux	3	B	S6	S8	0
F	Montaje y caracterización eléctrica	1	C	S6	S6	2
G	Adquisición y telemetría de sensores	2	F	S7	S8	2
H	Aplicación de agentes	2	D, E	S9	S10	0
I	Integración en la plataforma	2	G, H	S11	S12	0
J	Pruebas funcionales	1	I	S13	S13	0
K	Evaluación de precisión	1	J	S14	S14	0
L	Margen de contingencia	1	K	S15	S15	0
M	Preparación y presentación final	1	L	S16	S16	0

Hitos de control

El cuadro 1.4 define cinco hitos. Cada uno tiene un criterio de aceptación verificable y una acción predefinida en caso de incumplimiento.

Uso del margen de contingencia

La semana 15 se reserva como margen de contingencia y se activa cuando alguno de los hitos H3, H4 o H5 no se cumple en su fecha. Si el retraso acumulado supera una semana, se acuerda con el profesor asesor una reducción de alcance: se mantienen el recorrido UMBmark y la recta, y se elimina la prueba exterior con GPS.

Cuadro 1.4: Hitos de control del proyecto.

Hito	Criterio de aceptación	Acción en caso de incumplimiento
H1. Requerimientos (21 ago, S3)	El profesor asesor aprueba el documento de requerimientos, con valores numéricos para frecuencia, latencia, error e interfaz eléctrica.	El documento se corrige durante la semana 4, en paralelo con el diseño. Los valores faltantes se toman del anteproyecto y se registran como provisionales.
H2. Diseño (4 sep, S5)	El profesor asesor aprueba el esquemático, el diagrama de conexión y la arquitectura de agentes con sus mensajes.	El diseño eléctrico usa su holgura de dos semanas. La adaptación de CMAES comienza igual, porque no depende del detalle de los agentes.
H3. CMAES estable (25 sep, S8)	Se cumple el indicador del OE3 en la Raspberry Pi 5.	Se activa la contingencia de la sección 1.4.1 y el tiempo empleado se descuenta del margen de la semana 15.
H4. Sistema integrado (23 oct, S12)	Se cumple el indicador del OE5.	La corrección de la integración se hace en la semana 13 y las pruebas funcionales pasan a la semana 15.
H5. Evaluación (6 nov, S14)	Se cumple el indicador del OE6 en los tres escenarios.	La escala y el ancho de vía efectivo se recalibran en la semana 15 y las pruebas se repiten.

1.5. Enfoque de la solución

Esta sección describe cómo se exploró el espacio de diseño, cómo se evaluaron las alternativas y qué enfoque se eligió.

Se evaluaron cuatro alternativas, cada una con una combinación distinta de sensores y de algoritmo. Para compararlas se usó una matriz de decisión ponderada (cuadro 1.5) con cinco criterios: precisión esperada, complejidad de integración, carga de procesamiento, costo y parecido con las condiciones de una misión planetaria.

Alternativa A: solo odometría. Es la configuración actual de la plataforma. No cuesta nada y es la más simple, pero no resuelve ninguno de los problemas de la sección 1.2. Se usa como línea base para comparar las demás.

Alternativa B: odometría con corrección inercial. Agrega una IMU cuyo giroscopio mide la velocidad de giro del r  ver sin depender del contacto de las ruedas con el suelo. El error de la odomet  a crece con la distancia recorrida, mientras que el error del giroscopio crece con el tiempo. Como los dos errores tienen causas distintas, cada sensor puede corregir al otro. El costo es bajo y la carga de procesamiento cabe en la plataforma.

Alternativa C: odomet  a con correcci  n por sensores de distancia. Usa el LiDAR o los sensores ultras  nicos para medir la distancia a objetos del entorno. Esto da informaci  n absoluta sobre la posici  n, pero necesita un mapa previo o estructuras f  ciles de reconocer, y funciona peor en terreno natural sin estructuras.

Alternativa D: odomet  a con correcci  n visual. Es la t  cnica que usan los r  veres de exploraci  n m  s recientes [11, 12]. Es la que promete mayor precisi  n y tambi  n permite detectar el deslizamiento. Sin embargo, procesar im  genes requiere mucho c  mputo, y el controlador de alto nivel ya dedica buena parte de su capacidad al modelo de detecci  n de objetos [3]. Con esta alternativa no se cumplir  a el l  mite de latencia del subsistema.

Cuadro 1.5: Matriz de decisi  n ponderada. La alternativa A es la l  nea base y recibe 0 en todos los criterios.

Criterio	Peso	A	B	C	D
Precisi��n y robustez ante el deslizamiento	4	0	+2	+2	+2
Complejidad y tiempo de implementaci��n	3	0	��1	��2	��2
Carga de procesamiento adicional	2	0	0	0	��2
Costo de hardware nuevo	1	0	��1	��1	0
Parecido con una misi��n planetaria	1	0	0	��1	+1
Suma ponderada		0	4	0	��1

Se eligi   la alternativa B por tres razones. Primero, es la   nica que corrige el error de orientaci  n y detecta el deslizamiento sin necesitar infraestructura externa ni estructuras en el entorno. Segundo, su costo y su carga de procesamiento caben dentro de las restricciones del proyecto. Tercero, es la misma

configuración que se usa en misiones reales: en los r6veres de exploraci3n, la combinaci3n de odometr3a e IMU es la base sobre la que despu3s se agregan otras capas de percepci3n [13].

1.6. Alcances y limitaciones del proyecto

Este trabajo abarca el dise1o, la implementaci3n y la validaci3n experimental del subsistema de estimaci3n de pose en Olympus. Incluye la adaptaci3n de la biblioteca CMAES al Linux del controlador de alto nivel, la ampliaci3n del firmware del controlador de bajo nivel para leer la IMU y la construcci3n de un modelo de simulaci3n que sirve como base de comparaci3n.

Quedan fuera del alcance:

- **Usar el receptor GPS dentro del estimador.** El receptor no entra en el filtro por dos razones: mantener el parecido con una misi3n planetaria, el cual parte del prop3sito de Olympus, y porque el error de un receptor GPS es de varios metros, mucho mayor que el error permitido al estimador en las trayectorias previstas. Adem3s, ese error no es aleatorio de una muestra a otra, sino que cambia lentamente. El filtro de Kalman supone que el ruido de cada medici3n es independiente del anterior, as3 que usar el GPS romper3a una de sus hip3tesis b3sicas.
- **La estimaci3n de pose en seis grados de libertad.** El estimador trabaja en un plano y entrega tres grados de libertad: dos de posici3n y uno de orientaci3n. No estima la altura ni los 3ngulos de alabeo y cabeceo.
- **La correcci3n de la velocidad de giro por inclinaci3n.** El estimador supone que el r6ver se mueve sobre terreno horizontal. En pendientes pronunciadas, esto produce un error sistem3tico en la orientaci3n que esta versi3n no corrige.
- **La estimaci3n de pose con sensores externos,** como odometr3a visual, SLAM o navegaci3n por marcadores.
- **La certificaci3n formal del nivel de madurez tecnol3gica** seg3n normas espaciales.

Tambi3n se esperan tres limitaciones. La primera es de observabilidad. Sin una medici3n de posici3n absoluta, el error del estimador siempre crece con la distancia recorrida. Por eso el criterio de aceptaci3n se expresa como un porcentaje de esa distancia y no como un error fijo en metros. La segunda es de detecci3n. Comparar los encoders con el giroscopio permite detectar el deslizamiento cuando el r6ver gira, pero no cuando las seis ruedas patinan por igual y el r6ver no avanza. La tercera es de recursos. El proyecto dura diecisi3s semanas y lo realiza una sola persona, lo que limita la cantidad de pruebas experimentales.

1.7. Recursos y presupuesto

El proyecto usa recursos que ya existen en el SETEC Lab y algunos componentes comprados. La inversión directa calculada es de USD 64 y cubre la IMU, el receptor GPS, el traductor de niveles del OE1, los materiales de conexión y un margen de imprevistos de USD 20, equivalente al 45 % de las compras, financiado por el SETEC Lab. El margen es alto porque cubre el reemplazo de un módulo dañado durante las pruebas.+

A los recursos institucionales se les asigna un valor de referencia para mostrar el costo real del proyecto. El trabajo de ingeniería suma 320 horas (20 horas por semana durante 16 semanas); a un costo de referencia de USD 15 por hora, equivale a USD 4800. El costo total estimado es de USD 6809: USD 64 de inversión directa y USD 6745 de recursos institucionales valorados.

Cuadro 1.6: Presupuesto estimado del proyecto.

Recurso	Cant.	Unit. (USD)	Total (USD)	Observación
Róver Olympus	1	0	0	Activo del SETEC Lab
Raspberry Pi 5	1	125	125	Instalada; valor de referencia
Arduino Mega	1	45	45	Instalado; valor de referencia
Encoders Hall (NFP-5840-31ZY-EN)	6	10	60	Instalados; valor de referencia
IMU MPU-9250	1	8	8	Compra directa
Receptor GPS	1	18	18	Compra directa
GY-GPSV3-NEO				
Traductor de niveles	1 lote	3	3	Compra directa (OE1)
I ² C y resistencias				
Cables, conectores y protoboard	1 lote	15	15	Compra directa
Imprevistos y reemplazos	1 lote	20	20	45 % de las compras directas
Licencia de MATLAB y Simulink	1	330	330	Licencia institucional del TEC
Yocto Project y cadena de herramientas	1	0	0	Software libre
Biblioteca CMAES	1	0	0	Código del SETEC Lab
Uso de la infraestructura del SETEC Lab	40 h	10	400	Valor de referencia
Asesoría del profesor	32 h	30	960	Dos horas por semana
Trabajo de ingeniería	320 h	15	4800	Valor de referencia
Electricidad y servicios	16 sem.	–	25	Estimación
Total estimado			6809	Directa: 64; institucional: 6745

Capítulo 2

Marco teórico

Este capítulo presenta la teoría y los trabajos previos que sustentan el diseño del subsistema. Se tratan cuatro temas: la cinemática de vehículos que giran por deslizamiento, los errores de los sensores que miden el movimiento propio, la teoría de estimación de estado aplicada a la navegación y las arquitecturas de software multiagente para sistemas embebidos.

El capítulo tiene cuatro partes. La primera presenta los conceptos y las herramientas matemáticas. La segunda revisa cómo se ha resuelto el problema en el pasado. La tercera describe las técnicas más avanzadas que existen hoy. La cuarta compara este trabajo con esas técnicas y señala qué faltante concreto viene a cubrir.

En este capítulo, el filtro de Kalman extendido (sección 2.1.6) se presenta en su forma general. La forma específica para Olympus, con la elección de las variables de estado y la justificación de cada una, es una decisión de diseño y se desarrolla en la sección 3.2.

2.1. Fundamentos conceptuales

2.1.1. Autonomía y el problema de la localización

Un vehículo autónomo percibe su entorno, decide qué hacer y lo ejecuta sin que una persona intervenga todo el tiempo. Siegwart, Nourbakhsh y Scaramuzza [14] dividen esta capacidad en cuatro etapas que dependen una de otra: percepción, localización, planificación y control del movimiento. Este trabajo se ocupa de la localización. Sin saber dónde está el vehículo, la planificación no tiene un punto de partida y el control del movimiento no tiene con qué comparar la posición real con la deseada.

Esto conecta la localización con la teoría de control. La pose estimada es la señal de realimentación del lazo de navegación, así que lo que más importa para el sistema completo son dos propiedades: cada cuánto se actualiza y cuánto se tarda en calcularse. Un lazo de control realimentado tolera cierto retardo, pero

si el retardo es demasiado grande se reduce el margen de fase y el lazo puede volverse inestable [15, 16].

Hay dos formas de plantear la localización. La localización absoluta responde a la pregunta de dónde está el vehículo en un marco de referencia externo que ya existe. Para eso necesita observar algo de ese marco: satélites de navegación, balizas, marcadores visuales o un mapa previo. La localización relativa, también llamada navegación a estima o *dead reckoning*, responde a dónde está el vehículo respecto al punto donde empezó y se calcula sumando su propio movimiento paso a paso. La diferencia más importante está en el error. En la localización absoluta, el error no puede ser mayor que la precisión de la referencia externa. En la relativa, el error siempre crece con la distancia recorrida [17].

2.1.2. Restricciones de la exploración planetaria

Los ródveres planetarios trabajan bajo condiciones que descartan la mayoría de las soluciones de localización que se usan en la Tierra. En Marte y en la Luna no hay un sistema de navegación por satélite en funcionamiento, así que la posición se debe calcular a bordo [11]. Una señal de radio tarda hasta unos 22 minutos en viajar de la Tierra a Marte [18], según la posición de los planetas, y con ese retardo no es posible controlar el ródver en tiempo real desde la Tierra. Además, el suelo suelto (regolito) hace que las ruedas patinen de forma frecuente. Por último, las computadoras resistentes a la radiación que usan estas misiones son mucho más lentas que un procesador comercial actual, así que solo se pueden usar algoritmos cuyo costo de cálculo sea bajo y predecible [19].

Por todo esto, un ródver planetario tiene que estimar su pose principalmente por navegación a estima, con sensores que miden su propio movimiento, y complementarla con percepción del entorno cuando sea posible.

2.1.3. Sistemas de referencia y representación de la pose

Marcos de referencia

La pose de un vehículo solo tiene sentido si se indica respecto a qué marco de referencia se mide. En este trabajo se usan dos (figura 2.1):

- El **marco del cuerpo** $\{B\}$, que se mueve junto con el chasis. Su origen está en el centro geométrico del vehículo, el eje x_B apunta hacia el frente y el eje y_B hacia la izquierda. Los sensores inerciales miden en este marco.
- El **marco de navegación** $\{W\}$, fijo respecto al terreno. Su origen y su orientación se fijan en el momento en que se enciende el subsistema.

El marco de navegación merece una aclaración. El sistema no tiene un magnetómetro confiable ni otra referencia externa, así que no puede saber hacia dónde queda el norte ni ninguna otra dirección absoluta. El marco $\{W\}$ queda definido simplemente por la posición y la orientación que tiene el ródver cuando se enciende. Por lo tanto, la estimación responde a la pregunta dónde está el

róver respecto al punto y la orientación de partida. Para relacionar esa estimación con un mapa, hay que alinear el chasis con una referencia conocida antes de empezar cada prueba.

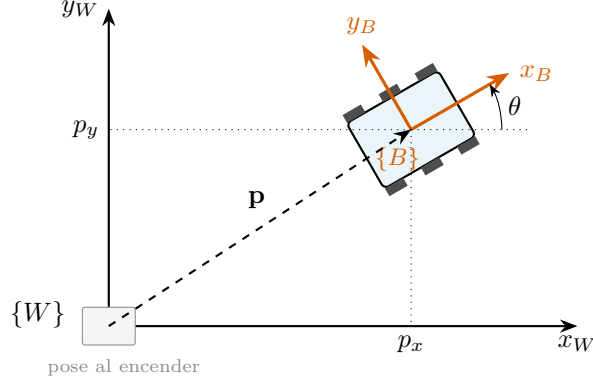


Figura 2.1: Marcos de referencia de navegaci  n $\{W\}$ y del cuerpo $\{B\}$, y pose del r  ver.

La pose en el plano y el grupo $SE(2)$

La pose de un cuerpo r  gido en el espacio tiene seis grados de libertad: tres de posici  n y tres de orientaci  n. En matem  tica, el conjunto de todas esas poses se llama grupo especial euclidiano $SE(3)$. Si el movimiento se limita a un plano, quedan tres grados de libertad (dos de posici  n y uno de orientaci  n), y el grupo se reduce a $SE(2)$. Una pose en $SE(2)$ se puede escribir como una matriz:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R}(\theta) & \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad \mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad \mathbf{p} = \begin{bmatrix} p_x \\ p_y \end{bmatrix}, \quad (2.1)$$

donde $\mathbf{R}(\theta)$ convierte coordenadas del marco del cuerpo al de navegaci  n [20]. En la implementaci  n se guardan solo los tres n  meros p_x , p_y y θ , en lugar de la matriz completa. De esta manera, se evita un problema num  rico: una matriz de rotaci  n debe cumplir ciertas condiciones (sus columnas deben tener largo uno y ser perpendiculares), y los redondeos del punto flotante las van rompiendo poco a poco.

Guardar el   ngulo θ como un n  mero tiene un costo: los   ngulos dan la vuelta. Por ejemplo, 179° y -179° est  n a solo 2° de distancia, pero su resta da 358° . Por esta raz  n, todo   ngulo que se calcula, y en especial toda diferencia entre dos   ngulos, se ajusta al intervalo $[-\pi, \pi)$ con la funci  n

$$\text{wrap}(\alpha) = ((\alpha + \pi) \bmod 2\pi) - \pi. \quad (2.2)$$

Por qué se trabaja en el plano

Pasar de $SE(3)$ a $SE(2)$ supone que el r  ver se mueve sobre un plano horizontal. Con esa suposici  n, la gravedad es perpendicular al plano de movimiento y no aparece en los ejes x_B ni y_B , lo que simplifica mucho el uso de los sensores inerciales.

La suposici  n es aceptable para este trabajo, pero tiene dos efectos que se deben tomar en cuenta. El primero es que la pose estimada es la proyecci  n del movimiento sobre el plano. Si el r  ver sube una rampa, la distancia medida en el plano es menor que la recorrida sobre el terreno. El segundo efecto es que el giroscopio mide la velocidad de giro en el marco del cuerpo, pero el modelo necesita la velocidad con que cambia el rumbo en el marco de navegaci  n. La relaci  n exacta entre ambas es

$$\dot{\psi} = \frac{\omega_y^B \sin \phi + \omega_z^B \cos \phi}{\cos \vartheta}, \quad (2.3)$$

donde ϕ y ϑ son los   ngulos de alabeo y cabeceo, y ω_y^B y ω_z^B son las velocidades angulares medidas en el cuerpo [21]. En terreno horizontal la ecuaci  n se reduce a $\dot{\psi} = \omega_z^B$. En cambio, con una pendiente de $\vartheta = 15^\circ$ y sin alabeo, usar solo ω_z^B da un rumbo que cambia un $1 - \cos 15^\circ \approx 3,4\%$ m  s lento que el real.

2.1.4. Cinem  tica del veh  culo

El modelo de unicycle y la restricci  n no holon  mica

Un veh  culo con ruedas convencionales no puede moverse de lado, porque sus ruedas solo ruedan hacia adelante o hacia atr  s. Esta condici  n se llama restricci  n no holon  mica: limita las velocidades posibles del veh  culo, pero no limita las posiciones a las que puede llegar (con maniobras, puede llegar a cualquiera) [22]. En el marco del cuerpo se escribe

$$v_y^B = 0. \quad (2.4)$$

Con esta restricci  n, el movimiento del veh  culo queda descrito por solo dos velocidades: la velocidad hacia adelante v y la velocidad de giro ω . Las ecuaciones de movimiento son las del llamado modelo de unicycle:

$$\dot{p}_x = v \cos \theta, \quad \dot{p}_y = v \sin \theta, \quad \dot{\theta} = \omega. \quad (2.5)$$

Este modelo tiene una ventaja que se aprovecha en la secci  n 3.2. Los estimadores que calculan la velocidad completa del veh  culo en el marco de navegaci  n tienen que imponer la restricci  n (2.4) como una medici  n adicional ficticia, llamada pseudomedida [23]. Con el modelo de unicycle esto no hace falta, porque el modelo mismo no permite el movimiento lateral.

Modelo diferencial

En un vehículo de tracción diferencial, v y ω se calculan a partir de las velocidades de dos ruedas, una a cada lado, separadas una distancia b :

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{b}. \quad (2.6)$$

Giro por deslizamiento y ancho de vía efectivo

Un vehículo que gira por deslizamiento (*skid steering*) tiene ruedas que no se orientan: las ruedas de un lado giran a distinta velocidad que las del otro, y el vehículo gira porque las llantas se arrastran de lado contra el suelo. En sentido estricto, esto viola la restricción (2.4), porque el deslizamiento lateral es justamente lo que produce el giro.

La solución más usada, validada con experimentos por Mandow et al. [9], es mantener la forma de (2.6) pero reemplazar el ancho de vía real b por un ancho de vía efectivo b_{eff} , más grande, que toma en cuenta el deslizamiento. Este parámetro depende de dónde quedan los centros instantáneos de rotación de cada lado del vehículo [24], y se escribe como

$$b_{\text{eff}} = \chi b_{\text{nom}}, \quad \chi > 1. \quad (2.7)$$

El factor χ depende del terreno, así que no se puede calcular a partir de la geometría del r  ver. Este debe ser medido con experimentos.

Forma discreta

El estimador no trabaja con velocidades continuas, sino con los incrementos que se miden en cada intervalo de muestreo Δt . Si se integra (2.5) en un intervalo, suponiendo que la velocidad de giro es constante, y se eval  a el rumbo en la mitad del intervalo, se obtiene

$$\begin{bmatrix} p_{x,k+1} \\ p_{y,k+1} \\ \theta_{k+1} \end{bmatrix} = \begin{bmatrix} p_{x,k} + \Delta s \cos(\theta_k + \frac{\Delta\theta}{2}) \\ p_{y,k} + \Delta s \sin(\theta_k + \frac{\Delta\theta}{2}) \\ \text{wrap}(\theta_k + \Delta\theta) \end{bmatrix}, \quad (2.8)$$

con $\Delta s = (\Delta s_R + \Delta s_L)/2$ y $\Delta\theta = (\Delta s_R - \Delta s_L)/b_{\text{eff}}$. Usar el rumbo de la mitad del intervalo, en lugar del rumbo al inicio, reduce el error que se comete en cada paso, pasa a ser proporcional a Δt^3 en lugar de Δt^2 .

La ecuaci  n (2.8) es el punto de partida del modelo de proceso del estimador.

2.1.5. Sensores y caracterizaci  n de sus errores

Encoders en cuadratura

Un encoder incremental en cuadratura entrega dos se  ales cuadradas, llamadas A y B, desfasadas 90 grados. El orden en que cambian las dos se  ales indica

el sentido de giro. Si se cuentan los flancos de subida y de bajada de ambas señales, se obtienen cuatro cuentas por cada ciclo del encoder (decodificación por cuatro). Otra opción es contar solo los flancos de la señal A y usar B para saber el sentido, lo que da dos cuentas por ciclo (decodificación por dos). La figura 2.2 compara ambas.

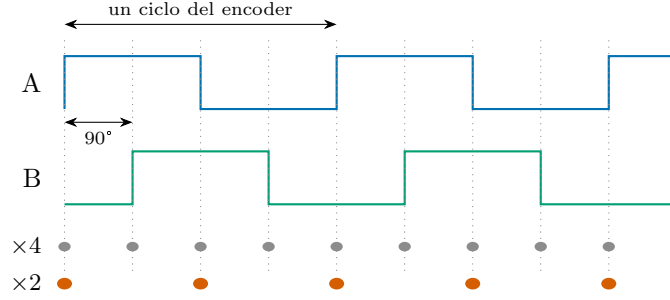


Figura 2.2: Señales de un encoder incremental en cuadratura y flancos contados con decodificación por cuatro y por dos.

La distancia que avanza una rueda por cada cuenta es

$$\lambda_i = \frac{2\pi r_i}{N_i}, \quad (2.9)$$

donde r_i es el radio de la rueda i y N_i es su número de cuentas por vuelta. N_i es el producto de las cuentas por vuelta del motor, la relación de reducción de la caja y el factor de decodificación. La resolución se define para cada rueda por separado porque las ruedas no tienen exactamente el mismo diámetro ni el mismo número de cuentas por vuelta, y esas diferencias son una fuente de error sistemático, como se explica más adelante en esta sección.

Cuantización y acumulación

La lectura de un encoder siempre es un número entero de cuentas. Si el error de redondeo se modela como uniforme dentro de una cuenta de tamaño λ , su varianza es

$$\sigma_{\text{cuant}}^2 = \frac{\lambda^2}{12}. \quad (2.10)$$

Hay un detalle de implementación que suele pasar desapercibido y que tiene consecuencias. Si el sistema envía incrementos ya redondeados en cada intervalo, la fracción de cuenta que sobra se pierde, y el error de redondeo se va sumando siempre en la misma dirección. El resultado es un error que crece con el tiempo. En cambio, si se envían las cuentas acumuladas y el receptor calcula la diferencia, la fracción que sobra se conserva para el siguiente intervalo, y el error total nunca supera una cuenta, sin importar cuántos intervalos pasen. Además, si se pierde una muestra, la siguiente trae el total completo y solo cambia la duración del intervalo.

Errores sistemáticos y no sistemáticos de la odometría

Borenstein y Feng [10] dividen los errores de la odometría en dos grupos, y esta división es la que se usa como referencia:

- **Errores sistemáticos.** Se repiten siempre igual y se pueden predecir. Los dominan dos parámetros: la relación entre los diámetros de las ruedas de ambos lados, E_d , y la relación entre el ancho de vía real y el que se supone, E_b . Se corrigen con calibración.
- **Errores no sistemáticos.** Los causan el deslizamiento, las irregularidades del terreno y la pérdida momentánea de contacto de las ruedas. No se pueden corregir con calibración. Se necesitan sensores adicionales u otra forma de detectarlos.

Giroscopios MEMS

Un giroscopio MEMS mide la velocidad angular, pero su medición incluye un sesgo que cambia lentamente y un ruido de banda ancha:

$$\omega_m = (1 + s)\omega + b_\omega + \eta, \quad \eta \sim \mathcal{N}(0, \sigma_g^2), \quad (2.11)$$

donde s es el error del factor de escala y b_ω es el sesgo. Estos términos se suelen caracterizar con la varianza de Allan [25], que permite separar el ruido blanco, la inestabilidad del sesgo y la deriva a largo plazo.

Hay dos propiedades del sesgo que afectan el diseño del estimador. La primera es que b_ω casi no cambia durante una prueba, así que se puede modelar como una caminata aleatoria de varianza pequeña. La segunda es que depende de la temperatura, así que puede cambiar durante una operación larga en la que la electrónica de potencia cercana se calienta. La sección 3.2.8 analiza qué implica esta segunda propiedad.

Cuando se integra la medición del giroscopio para obtener el rumbo, el error crece con el tiempo y no con la distancia, porque el término que domina es el sesgo que queda sin corregir multiplicado por el tiempo de integración. Esta es la razón por la que la odometría y el giroscopio se complementan: el error de la primera depende de la distancia y el del segundo depende del tiempo. En esta idea se basa la técnica llamada *gyrodometry*, propuesta por Borenstein y Feng [26].

Acelerómetros

Un acelerómetro mide la llamada fuerza específica, es decir, la aceleración del sensor menos la parte de la gravedad que cae sobre sus ejes. Para obtener velocidad hay que integrar su medición una vez, y para obtener posición, dos veces. De ahí que cualquier sesgo que no se corrija produzca un error de posición que crece con el cuadrado del tiempo, o con el cubo si además hay error en la orientación [27].

En un vehículo lento, como un rover de exploración, la señal útil del acelerómetro es muy pequeña comparada con las perturbaciones. Esto es porque la aceleración propia del rover es del mismo tamaño que el ruido del sensor. Por otro lado, una inclinación de apenas 1° hace que la gravedad aporte al eje longitudinal $g \sin(1^\circ) \approx 0,17 \text{ m s}^{-2}$, varias veces más que toda la aceleración que el rover produce al moverse. En la práctica, en un vehículo así el acelerómetro se puede utilizar para medir la inclinación, pero no la aceleración.

Sistemas globales de navegación por satélite

Un receptor GNSS (*Global Navigation Satellite System*) entrega la posición absoluta con un error que no crece con el tiempo, así que en principio podría limitar la deriva de la navegación a estima. Tiene, sin embargo, dos limitaciones importante: un receptor de bajo costo sin corrección diferencial tiene un error de varios metros [27] y ese error no es independiente de una muestra a la siguiente, sino que se mantiene parecido durante segundos o minutos. El filtro de Kalman supone lo contrario, así que si se le entrega una medición así termina confiando en ella más de lo debido [27]. Además, en otros planetas no hay navegación por satélite (sección 2.1.2), por lo que un estimador que dependa del GNSS no reproduce las condiciones de una misión planetaria.

2.1.6. Estimación de estado

Formulación bayesiana recursiva

Sea $\mathbf{x}_k$ el estado del sistema en el instante k , $\mathbf{u}_k$ una entrada conocida y $\mathbf{z}_k$ una medición. El modelo general es:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k), \quad (2.12)$$

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{\Sigma}_v). \quad (2.13)$$

El problema de filtrado consiste en calcular la distribución de probabilidad del estado actual dado todo lo medido hasta ahora, $p(\mathbf{x}_k | \mathbf{z}_{1:k})$. Si se supone que el estado siguiente depende solo del estado actual (hipótesis de Markov), esa distribución se puede calcular de forma recursiva en dos pasos, predicción y actualización, sin volver a procesar todas las mediciones anteriores [28]. Esta propiedad es la que permite hacer la estimación en tiempo real en un sistema embebido.

El filtro de Kalman

Si f y h son lineales y los ruidos son gaussianos, la distribución del estado sigue siendo gaussiana en todo momento, y basta con conocer su media y su covarianza. El filtro de Kalman [29] calcula esas dos cantidades paso a paso y, bajo esas condiciones, es el estimador de mínima varianza [30].

El filtro de Kalman extendido

El modelo (2.8) no es lineal, porque contiene senos y cosenos del estado. El filtro de Kalman extendido (EKF) resuelve esto aproximando f y h con rectas alrededor de la estimación actual, usando sus jacobianos:

$$\mathbf{F}_k = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1}}, \quad \mathbf{H}_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_k^-}. \quad (2.14)$$

Cada ciclo del filtro tiene estos pasos (figura 2.3):

$$\hat{\mathbf{x}}_k^- = f(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k), \quad \mathbf{P}_k^- = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}_k, \quad (2.15)$$

$$\mathbf{y}_k = \mathbf{z}_k - h(\hat{\mathbf{x}}_k^-), \quad \mathbf{S}_k = \mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^\top + \Sigma_v, \quad (2.16)$$

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^\top \mathbf{S}_k^{-1}, \quad \hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \mathbf{y}_k. \quad (2.17)$$

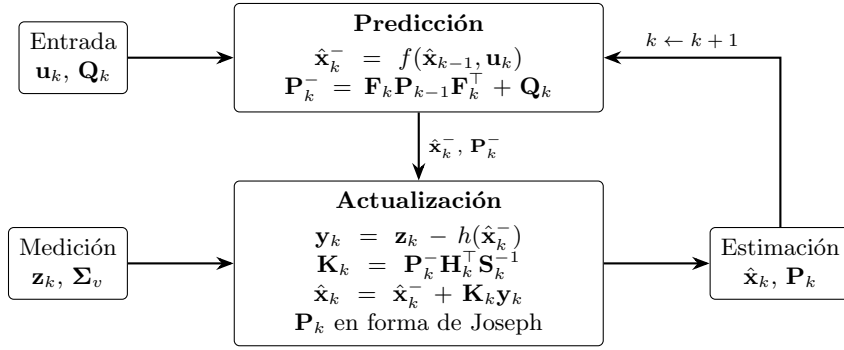


Figura 2.3: Ciclo de predicción y actualización del filtro de Kalman extendido.

Se eligió el EKF en lugar de otras opciones no lineales, como el filtro de Kalman *unscented* [31], por dos razones. El modelo (2.8) es poco no lineal y su jacobiano se puede escribir de forma exacta, así que el error de la aproximación lineal es pequeño. Además, el EKF necesita mucho menos cálculo, lo cual importa en una plataforma embebida.

Forma de Joseph

La covarianza se suele actualizar con la fórmula corta $\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^-$. Esta fórmula solo es correcta si $\mathbf{K}_k$ es exactamente la ganancia óptima. Con los redondeos del punto flotante, puede producir matrices que dejan de ser simétricas o que tienen valores propios negativos, y entonces el filtro se degrada o diverge. La forma de Joseph,

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^- (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^\top + \mathbf{K}_k \Sigma_v \mathbf{K}_k^\top, \quad (2.18)$$

es correcta para cualquier ganancia y mantiene la matriz simétrica y semidefinida positiva por construcción, a cambio de más operaciones [32, 33]. Se usa en

este trabajo porque es más robusta numéricamente, lo cual importa porque el estimador final va a correr en un sistema embebido.

Observabilidad

Un estado es observable si su valor se puede determinar a partir de las entradas y las mediciones durante un tiempo finito. En los sistemas no lineales, la observabilidad es local: depende del punto de operación [20].

En este trabajo la observabilidad decide qué variables se pueden incluir en el vector de estado. Si se incluye un estado que no es observable, ese estado no se queda quieto. Absorbe errores de otras fuentes y empeora la estimación. La sección 3.2.8 muestra un caso concreto en el que un estado no observable absorbe un error de calibración y anula el aporte de un sensor.

Consistencia del filtro

Un estimador es consistente cuando la covarianza que reporta describe bien el tamaño real de su error. La prueba estándar usa el error cuadrático normalizado del estado (NEES):

$$\varepsilon_k = (\mathbf{x}_k - \hat{\mathbf{x}}_k)^\top \mathbf{P}_k^{-1} (\mathbf{x}_k - \hat{\mathbf{x}}_k). \quad (2.19)$$

Si el filtro es consistente, el valor esperado de ε_k es igual al número de variables que se comparan [32]. Si el valor es mayor de forma sistemática, el filtro es demasiado optimista, entonces dice tener menos incertidumbre de la que tiene, y eso es más peligroso que el error mismo. Si es menor, el filtro es demasiado pesimista y desaprovecha información.

Cuando la verdad de terreno solo mide la posición, la prueba se aplica al bloque de posición de $\mathbf{x}$ y de $\mathbf{P}$, y el valor esperado de ε_k es 2. Aun así, es un criterio de aceptación más exigente que solo medir el error.

2.1.7. Arquitecturas de software para sistemas embebidos

Arquitectura de dos capas

Muchas plataformas robóticas móviles usan dos computadoras. Un controlador de bajo nivel, con pocos recursos pero con tiempos de respuesta predecibles, que maneja directamente los sensores y los actuadores. Un controlador de alto nivel, con más capacidad, se encarga de la percepción y de la toma de decisiones [14].

El paradigma multiagente

Un sistema multiagente está formado por entidades independientes, llamadas agentes. Cada agente guarda su propio estado y su propio comportamiento, y los agentes se coordinan enviándose mensajes, en lugar de compartir memoria [34]. Cada agente tiene su propio ciclo de vida y un buzón donde recibe mensajes, y la plataforma que los ejecuta se encarga de registrarlos y de planificar cuándo corre cada uno [35]. En satélites pequeños, este enfoque se ha usado para dividir

el software de a bordo en partes independientes y para detectar y aislar fallos de sensores [4].

Para un sistema embebido, lo más útil de este paradigma no es que los agentes sean autónomos, sino la disciplina de comunicación que impone. Como los agentes no pueden compartir datos de forma escondida, toda dependencia entre componentes aparece de forma explícita en el diseño y se puede verificar en la implementación. La sección 2.2.4 describe el origen de la biblioteca que usa este trabajo.

2.1.8. Métodos de validación

El procedimiento UMBmark

Borenstein y Feng [36] propusieron un procedimiento estándar para medir los errores sistemáticos de la odometría. El vehículo recorre un cuadrado en sentido horario y luego en sentido antihorario, y se compara la posición final estimada con la real. El procedimiento está pensado para detectar errores que se compensan entre sí y que pasarían desapercibidos en pruebas más simples. Comparar los dos sentidos de recorrido permite separar el efecto de E_d del de E_b (figura 2.4a).

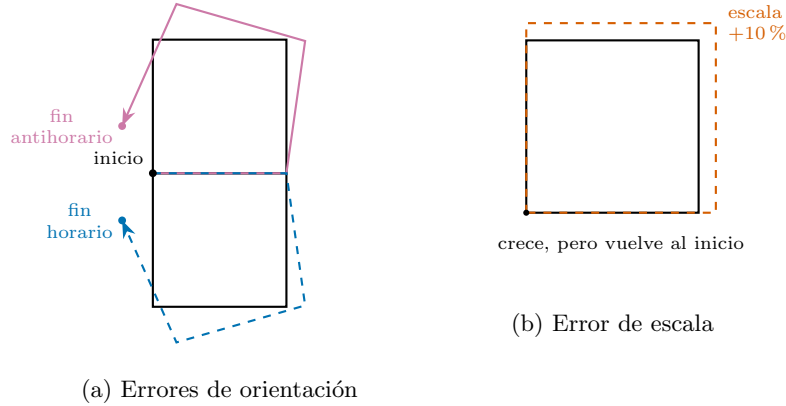


Figura 2.4: Procedimiento UMBmark: (a) errores de orientación y (b) error de escala. Basado en [36].

Este procedimiento tiene una propiedad que afecta el diseño de las pruebas. Si todas las distancias tienen el mismo error de escala (por ejemplo, un radio de rueda 2 % más grande que el real), el cuadrado completo sale 2 % más grande, pero sigue cerrando, y el error no se ve en la posición final. UMBmark detecta errores de orientación, no de escala (figura 2.4b). Para medir la escala se necesita una prueba en línea recta con la distancia medida de forma independiente. Las dos pruebas se complementan y ninguna reemplaza a la otra.

Verdad de terreno

Para medir el error del estimador se necesita una referencia externa bastante más precisa que la cota que se quiere verificar.

Como el error de la navegación a estima crece con la distancia, el criterio de aceptación se expresa como un porcentaje de la distancia recorrida y no como un error fijo en metros. Lo que interesa no es cuánto se equivoca el estimador en un punto dado, sino qué tan rápido acumula error.

2.2. Revisión de literatura

Esta sección repasa cómo se han resuelto antes los dos problemas principales de este trabajo, que son cómo estimar la pose de un vehículo sin una referencia absoluta y cómo organizar el software embebido que hace esa estimación.

2.2.1. De la odometría pura a la fusión inercial

La navegación a estima con odometría de ruedas es la técnica más antigua y sigue siendo la base de las demás. Su estudio sistemático se consolidó con los trabajos de Borenstein y sus colaboradores en la década de 1990. Su aporte más importante fue separar los errores sistemáticos de los no sistemáticos [10, 17]. Los primeros se repiten siempre igual y se pueden corregir con calibración. Los segundos, causados por el deslizamiento y las irregularidades del terreno, no.

Esa separación tuvo dos consecuencias. La primera fue el procedimiento UMBmark [36], que dio un método reproducible para medir los errores sistemáticos y que todavía es la referencia en el tema, y la segunda fue reconocer que los errores no sistemáticos necesitan sensores adicionales, lo que llevó directamente a usar sensores inerciales.

La técnica de *gyrodometry* [26] formalizó la idea en la que se basa este trabajo. El error de la odometría crece con la distancia y el de un giroscopio integrado crece con el tiempo. Ninguno de los dos sensores limita por sí solo el error del otro, pero como sus errores tienen causas distintas, una buena combinación de ambos es mejor que cualquiera por separado. En la misma época, Barshan y Durrant-Whyte [37] describieron cómo se comportan las unidades inerciales de bajo costo en robótica móvil, y midieron los niveles de sesgo y deriva que limitan su uso.

Más adelante se empezó a aprovechar el propio comportamiento del vehículo como fuente de información. Dissanayake et al. [23] mostraron que la restricción no holonómica de un vehículo terrestre se puede usar como una pseudomedida, sin necesitar ningún sensor adicional, y que con eso se reduce mucho la deriva lateral de una IMU de bajo costo. En la misma familia de técnicas están las actualizaciones de velocidad cero y de velocidad angular cero, que se aplican cuando el vehículo está quieto [27]. La sección 3.2.8 explica por qué estas actualizaciones son importantes en este trabajo.

2.2.2. Cinemática de vehículos que giran por deslizamiento

Modelar un vehículo que gira por deslizamiento tiene una dificultad de fondo: su forma de girar viola la restricción no holonómica en la que se basan los modelos cinemáticos comunes. La literatura resolvió el problema con experimentos. Martínez et al. [24] propusieron describir el vehículo por la posición de los centros instantáneos de rotación de cada oruga, medidos de forma experimental. Mandow et al. [9] extendieron la idea a vehículos con ruedas y comprobaron que un modelo diferencial con un ancho de vía efectivo, medido con experimentos, reproduce el comportamiento real con precisión suficiente para navegar.

La conclusión general de estos trabajos es que el parámetro que relaciona la odometría con el giro en este tipo de vehículo no se puede calcular a partir de la geometría, hay que medirlo y cambia según el terreno.

2.2.3. Estimación de pose en róveres de exploración

La historia de los róveres marcianos muestra cómo evolucionó el problema (figura 2.5). Sojourner navegaba de forma reactiva y usaba odometría de ruedas [38]. Spirit y Opportunity agregaron odometría visual, que usaban solo en algunos tramos por su alto costo de cálculo, y una IMU que servía sobre todo para medir su orientación [13, 11].

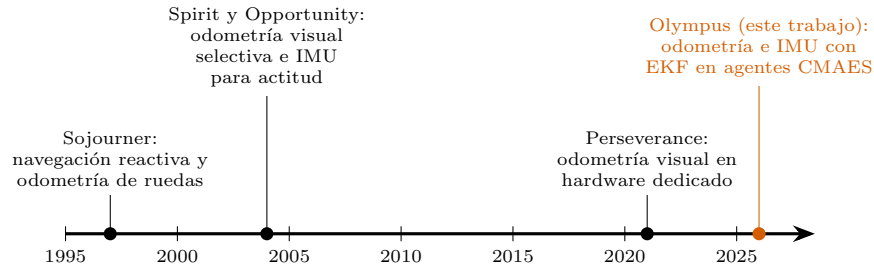


Figura 2.5: Evolución de la estimación de pose en róveres de exploración planetaria. Basado en [38, 13, 11, 12].

Maimone et al. [11] reportaron el resultado más citado de esa experiencia. La odometría visual permitió detectar y corregir deslizamientos que la odometría de ruedas no mostraba y fue decisiva para operar en pendientes. En casos extremos, el róver no avanzaba nada aunque las ruedas giraban con normalidad. Después, la investigación en terramecánica [39, 40] estudió la física de esa interacción entre la rueda y el suelo.

La lección que orienta este trabajo es que, en la navegación a estima de un róver, el error principal no viene del ruido de los sensores. Viene de los errores sistemáticos que causan los parámetros cinemáticos mal calibrados y del deslizamiento que el modelo no contempla.

2.2.4. Arquitecturas de software multiagente para sistemas embebidos

El segundo tema de este trabajo es la organización del software. Un sistema multiagente está formado por entidades independientes que guardan su propio estado y comportamiento, y que se coordinan enviándose mensajes [34]. La organización FIPA estandarizó la estructura de esos mensajes [41] y el ciclo de vida de los agentes [35]. Sobre esas normas se construyeron plataformas de uso general como JADE [42].

Estas plataformas necesitan más recursos y toleran más latencia de lo que permite un sistema embebido de tiempo real, lo que dio origen a una línea de trabajo específica. Carvajal-Godínez propuso una metodología para diseñar el software de a bordo de satélites pequeños como un sistema multiagente, y la aplicó a la detección y recuperación de fallos en giroscopios [4, 43]. A partir de esa metodología, el marco MAES [5, 6] llevó los agentes a sistemas embebidos, donde cada agente corre como una tarea del sistema operativo de tiempo real y cada buzón es una cola del núcleo. Rojas Marín [7] creó FreeMAES, que adapta MAES a FreeRTOS, y Conejo Boza [8] desarrolló CMAES, una versión en lenguaje C para la computadora espacial NanoMind A3200.

2.3. Estado del arte

2.3.1. Odometría visual e inercial y SLAM

Hoy, las técnicas más avanzadas para estimar la pose de un vehículo sin referencia absoluta combinan cámaras con sensores inerciales. Los sistemas de odometría visual-inercial estiman al mismo tiempo la pose, la velocidad y los sesgos de la IMU siguiendo puntos característicos de las imágenes [44, 45]. Estos acumulan mucho menos error que la fusión de sensores propios. El siguiente paso es el SLAM (localización y mapeo simultáneos), que además construye un mapa del entorno. Cuando el vehículo vuelve a un lugar conocido, el SLAM reconoce ese lugar y corrige el error acumulado, así que el error deja de crecer sin límite [46]. En Perseverance, la odometría visual y la navegación autónoma corren sobre hardware dedicado, lo que le permite recorrer distancias mucho mayores que sus antecesores [12].

2.3.2. Otras formas de construir el estimador

La literatura reciente también ha explorado alternativas al filtro de Kalman extendido. Los métodos basados en grafos de factores plantean la estimación como un problema de mínimos cuadrados sobre una ventana de estados pasados [47]. Así pueden volver a linealizar y corregir estimaciones anteriores, pero necesitan más memoria y más cálculo. Los filtros invariantes aprovechan la estructura matemática del espacio de poses (un grupo de Lie) para que su convergencia no dependa de la trayectoria estimada, y así evitan algunos problemas de consistencia del EKF común [48]. El filtro de Kalman *unscented* [31]

evita calcular jacobianos: en lugar de linealizar, propaga un conjunto de puntos elegidos (puntos sigma) a través del modelo no lineal.

2.3.3. Tendencias en cómputo y software a bordo

La tendencia en el cómputo de los vehículos de exploración es usar arquitecturas heterogéneas. En ellas, aceleradores dedicados se encargan de la percepción con tiempos de respuesta predecibles, y el procesador principal se encarga de la lógica de la misión [12, 19]. En el software, la práctica más común hoy es organizar el sistema en componentes con interfaces claras que se comunican por mensajes, porque esto facilita la verificación y evita que una falla en un componente se propague a los demás. El paradigma multiagente que usa este trabajo sigue esa tendencia, y además tiene un modelo formal de ciclo de vida y de comunicación basado en un estándar [41, 35].

2.4. Síntesis y posicionamiento del subsistema

Estas técnicas del estado del arte tienen algunos obstáculos para Olympus. El rover ya cuenta con la capacidad de detectar objetos, pero para poder utilizar modelos odometría visual requiere más capacidad de procesamiento y una cámara adecuada (ocupa un campo de visión mayor o igual a 120 y otras características). Ya el controlador de alto nivel de Olympus usa su capacidad en el subsistema de navegación y tiene una cámara funcional para esto, por lo que ninguna de estas opciones es mejor para Olympus. El modelo cinemático en el plano es poco no lineal y su jacobiano se conoce de forma exacta, así que el error de linealización del EKF es pequeño. No es necesario utilizar otro método de fusión más complejo o que consuma más recursos. Además, en esta plataforma el error principal viene de la calibración de los parámetros y no de la linealización, así que el cálculo extra de las alternativas no se traduciría en una mejora proporcional.

El cuadro 2.1 compara el estado del arte con la solución de este trabajo y explica cada diferencia.

El cuadro no quiere decir que este subsistema sea una versión reducida del estado del arte, sino que el estado del arte no se puede aplicar con las restricciones de Olympus. Sin una cámara disponible para la estimación, sin capacidad de cálculo libre y sin una referencia absoluta aceptable, la fusión de sensores propioceptivos no es una opción entre varias: es la única que se puede usar. El aporte de este trabajo es llevar esa solución tan lejos como lo permitan los sensores disponibles, y medir con claridad dónde está su límite.

Olympus tiene el hardware de odometría, la interfaz para una IMU y una computadora capaz de ejecutar el algoritmo, pero no tiene la fusión que une todo esto. Su propia documentación registra como pendiente el requisito de precisión de navegación, y los campos de pose de la telemetría se envían en cero [3]. El aporte de este proyecto es cubrir ese faltante usando el paradigma multiagente desarrollado en el laboratorio.

Cuadro 2.1: Posicionamiento del subsistema de estimación de pose respecto al estado del arte.

Aspecto	Estado del arte	Este trabajo	Razón de la diferencia
Sensores	Cámara e IMU combinadas; LiDAR [45, 46]	Encoders y giroscopio	El procesador del HLC ya está ocupado con la percepción.
Estimador	Grafos de factores; filtros invariantes [47, 48]	EKF con pocas variables de estado	Modelo poco no lineal; el error principal viene de la calibración.
Límite del error	Corrección al volver a lugares conocidos (SLAM) [46]	Sin límite; el error se mide.	No hay referencia absoluta, igual que en una misión planetaria.
Referencia absoluta	GNSS con corrección RTK o marcadores	Ninguna dentro del filtro; GNSS solo para validar	Parecido con una misión planetaria y precisión insuficiente del receptor
Software a bordo	Componentes que se comunican por mensajes	Agentes CMAES sobre hilos POSIX	Continuidad con la línea de investigación del SETEC Lab
Validación	Conjuntos de datos públicos de referencia [49, 50]	UMBmark en un análogo terrestre	No existen conjuntos de datos para esta plataforma.

El capítulo 3 documenta el desarrollo del subsistema, y en él se aplican estas herramientas al caso de Olympus.

Capítulo 3

Desarrollo del subsistema de estimación de pose

Este capítulo documenta el diseño del subsistema de estimación de pose, así como todos los pasos intermedios que se desarrollaron para lograr su integración. Las secciones siguen el orden de dependencia del trabajo. Primero se presenta la adaptación de la biblioteca CMAES a Linux (sección 3.1), que es la base de todo el software del controlador de alto nivel. Luego se presentan el diseño del estimador (sección 3.2) y el modelo de simulación con el que se diseñó y se evalúa (sección 3.3). Después se describe el diseño de los agentes del controlador de alto nivel, que integra el estimador en la arquitectura multiagente (sección 3.4). Por último se presenta el diseño preliminar de la interfaz eléctrica de la IMU (sección 3.5).

3.1. Adaptación de la biblioteca CMAES a Linux

La arquitectura del subsistema necesita componentes independientes que corran al mismo tiempo (adquisición, fusión de datos, estimación y comunicación) y que se comuniquen por mensajes, sin compartir memoria. CMAES ya ofrece esa estructura: agentes, buzones, un sistema de gestión de agentes y comportamientos [8]. El problema es que CMAES solo funcionaba sobre FreeRTOS, y el controlador de alto nivel de Olympus es una opera con Linux. Antes de escribir cualquier agente del subsistema fue necesario darle a CMAES una base que funcionara directamente sobre el sistema operativo objetivo. Esta sección describe ese trabajo, desde el análisis de la biblioteca original, la elección de la estrategia, el diseño, la implementación y las pruebas, incluidos los errores que se encontraron en el camino. La figura ?? resume el proceso.

Para eso, este trabajo lleva CMAES a GNU/Linux mediante una capa de portabilidad basada en hilos POSIX. Cambiar el núcleo de tiempo real por un

planificador de uso general tiene una consecuencia importante: ya no se puede garantizar cuánto tarda en activarse cada agente. Esto afecta los casos peores de la latencia de extremo a extremo, aunque no necesariamente su valor promedio.

3.1.1. Análisis de la biblioteca original

CMAES es la versión en lenguaje C de FreeMAES [7, 8], que a su vez implementa el marco MAES de Chan-Zheng [5]. Como C no tiene clases, la biblioteca las imita. Cada “clase” es una estructura que guarda sus datos junto con punteros a sus funciones, y unas funciones llamadas **Constructor*** llenan esos punteros al crear cada objeto. Sus componentes principales, organizados como muestra la figura 3.1, son:

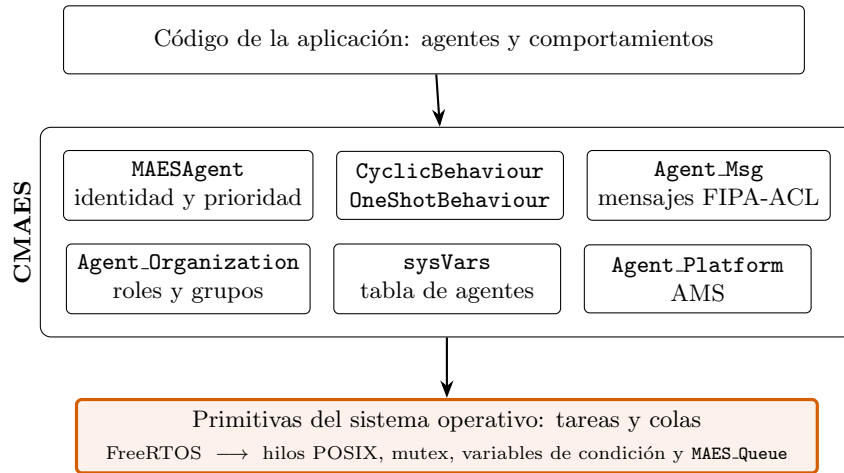


Figura 3.1: Estructura de capas de la biblioteca CMAES.

- **sysVars**: una tabla global de 64 entradas que relaciona el identificador de cada tarea con su agente. Cumple la función de un almacenamiento local por tarea.
- **Agent_Msg**: la mensajería. El buzón de cada agente es una cola de un solo espacio que bloquea al emisor cuando está llena. Los mensajes llevan un tipo al estilo de FIPA-ACL (REQUEST, INFORM, PROPOSE, entre otros) y un puntero al contenido [41].
- **Agent_Platform**: el sistema de gestión de agentes (AMS). Es un agente con privilegios que atiende solicitudes para registrar, suspender, reanudar, eliminar y reiniciar agentes. Cada solicitud pasa por unas condiciones que define el usuario.

- **CyclicBehaviour** y **OneShotBehaviour**: los comportamientos. Ejecutan una vez la función de preparación y luego repiten la acción del agente hasta que se cumple su condición de término.
- **Agent_Organization**: grupos de agentes con roles, que limitan qué agentes pueden enviarse mensajes entre sí.

Antes de diseñar nada, se buscó en todo el código qué archivos llaman funciones de FreeRTOS (`xTaskCreate`, `xQueueSend`, `vTaskSuspend` y similares). Solo tres lo hacen: `Agent_Platform.c`, `Agent_Msg.c` y `Agent_Organization.c`. Los demás archivos son portables sin ninguna dependencia del sistema operativo.

Una segunda observación confirmó este resultado. El repositorio ya tenía una segunda versión incompleta de la biblioteca, `libCMAES_CSP`, pensada para la pila de comunicaciones satelitales `libcsp`. Al comparar esa versión con la de FreeRTOS, las diferencias estaban exactamente en los mismos tres archivos. Dos señales independientes indicaban que la parte de la biblioteca que depende del sistema operativo es pequeña y se puede reemplazar sin tocar el resto.

3.1.2. Elección de la estrategia

Con ese resultado quedaban tres opciones realistas, resumidas en el cuadro 3.1.

Cuadro 3.1: Estrategias evaluadas para llevar CMAES a Linux.

Opción	En qué consiste	Ventajas y desventajas
Simulador POSIX de FreeRTOS	Compilar la versión de FreeRTOS casi sin cambios sobre el puerto oficial de FreeRTOS para GCC y POSIX	Es la opción que requiere menos esfuerzo, pero las tareas siguen siendo tareas simuladas de FreeRTOS dentro de un solo proceso de Linux, no hilos reales.
Terminar la versión para <code>libcsp</code>	Completar <code>libCMAES_CSP</code> , que ya tiene un controlador para hilos POSIX	Reutiliza más código, pero agrega una pila de red satelital que no hace falta para enviar mensajes dentro de un solo proceso.
Base nativa con hilos POSIX (elegida)	Implementar la misma interfaz <code>CMAES.h</code> directamente sobre hilos, mutex y variables de condición de POSIX	Requiere más diseño al inicio, pero usa hilos reales con la planificación de Linux y no tiene capas de emulación.

Se eligió la base nativa con hilos POSIX. Da hilos reales de Linux con planificación real, que importa para los requisitos de latencia del subsistema, y no agrega capas de emulación ni una pila de red innecesaria. El costo de esta opción es escribir desde cero la parte que depende del sistema operativo, en lugar de reutilizar una de las dos versiones existentes.

3.1.3. Diseño

Correspondencia de tipos

El objetivo era que la interfaz pública de `CMAES.h` no cambiara, mismas estructuras de agentes, mismos tipos de mensaje y mismas funciones. Solo cambian las primitivas de más abajo (cuadro 3.2). Así, una aplicación escrita para CMAES sobre FreeRTOS se puede compilar en Linux sin modificarla.

Cuadro 3.2: Correspondencia entre las primitivas de FreeRTOS y las de la nueva base.

FreeRTOS	Base con hilos POSIX
<code>xTaskHandle</code> (identificador de agente)	<code>pthread_t</code>
<code>xQueueHandle</code> (buzón)	<code>MAES_Queue*</code> (tipo nuevo)
Ticks del sistema operativo	Milisegundos en <code>uint32_t</code>
<code>portMAX_DELAY</code>	<code>MAES_MAX_DELAY</code> (valor <code>UINT32_MAX</code>)
<code>xTaskGetCurrentTaskHandle()</code>	<code>MAES_GetCurrentTaskHandle()</code> , que llama a <code>pthread_self()</code>

El buzón

En la biblioteca original, cada buzón es una cola de FreeRTOS de un solo espacio, que bloquea con un tiempo límite. La nueva base la reemplaza con un tipo propio, `MAES_Queue`: un mutex y dos variables de condición, una que avisa que el buzón tiene un mensaje y otra que avisa que tiene espacio. Si el emisor encuentra el buzón lleno, espera hasta que se vacíe. Un tiempo límite finito se convierte en una fecha límite absoluta para `pthread_cond_timedwait`, y un tiempo límite de cero significa intentar una vez sin esperar. La figura 3.2 muestra la secuencia. La espera está protegida por una función de limpieza, de modo que si el hilo se cancela mientras espera, el mutex no queda bloqueado.

El buzón copia la estructura del mensaje, pero esa estructura solo contiene un puntero al contenido, no el contenido. El agente que envía debe mantener válido ese contenido hasta que el receptor termine de leerlo. Por eso, cada agente emisor guarda sus mensajes en un conjunto fijo de búferes que usa por turnos. En un buzón de profundidad 1 pueden estar ocupados al mismo tiempo tres búferes: el que el receptor está leyendo, el que espera en el buzón y el que el emisor está llenando con el dato siguiente. Con solo dos búferes, el emisor terminaría escribiendo sobre uno de los dos primeros. Por lo tanto, la rotación necesita al menos tres búferes (figura 3.3).

Ciclo de vida de los agentes

FreeRTOS permite suspender, reanudar o eliminar una tarea desde otra tarea en cualquier momento. POSIX no ofrece nada equivalente. Para reproducir el mismo comportamiento, cada agente se crea como un hilo real que ya está

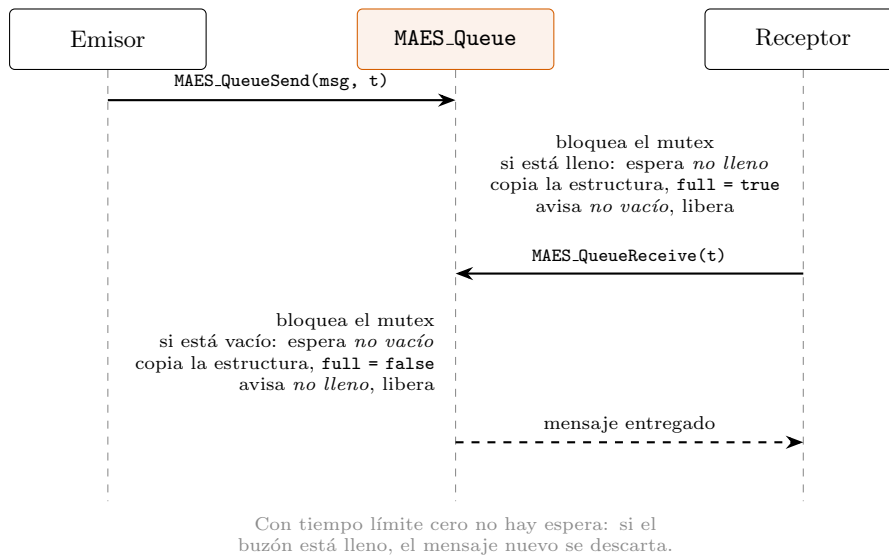


Figura 3.2: Secuencia de envío y recepción en el buzón MAES.Queue.

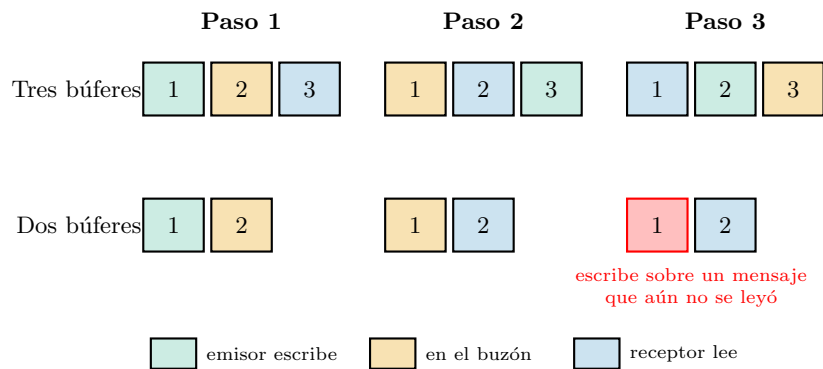


Figura 3.3: Rotación de búferes con un buzón de profundidad 1.

corriendo, pero lo primero que hace es detenerse en una “puerta de inicio” propia. La puerta se abre cuando el AMS registra o reanuda al agente, y se vuelve a cerrar cuando lo suspende. Con un solo mecanismo se cubren el agente recién creado, que espera a ser registrado, y todas las suspensiones y reanudaciones posteriores (figura 3.4).

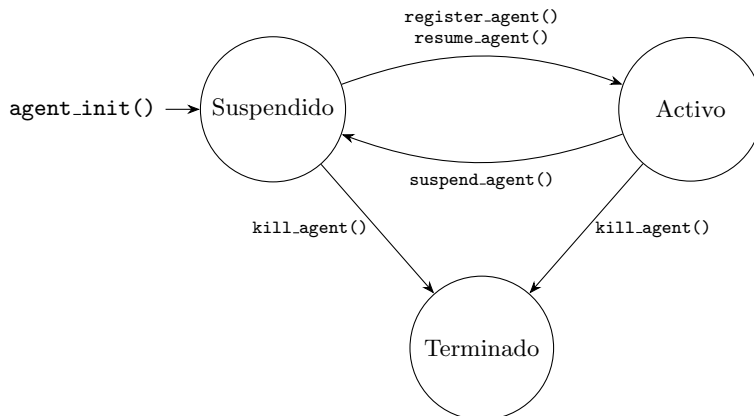


Figura 3.4: Ciclo de vida de un agente en la nueva base de CMAES.

Como POSIX no permite detener otro hilo desde afuera, la suspensión es cooperativa. El agente no se detiene en el instante en que se pide, sino la próxima vez que pasa por un punto de control, como la función `MAES_CheckSuspend()`, que se llama al inicio de cada iteración de los comportamientos y dentro de las esperas del agente. Esta es la única diferencia de comportamiento respecto a FreeRTOS que se debe tener en cuenta al escribir agentes. Una suspensión se aplica al terminar la iteración en curso, no de inmediato.

Para eliminar un agente se usan `pthread_cancel()` y `pthread_join()` con cancelación diferida. Así, el hilo solo se detiene en puntos definidos, como la espera del buzón, y nunca a mitad de una actualización de datos compartidos.

El arranque sigue este orden. El programa principal construye la plataforma, que guarda el identificador del hilo que la creó. Luego inicializa cada agente, cuyo hilo arranca y se detiene en su puerta de inicio. Después crea el hilo del AMS, que también espera en su puerta. Por último, el AMS se registra a sí mismo y registra a los demás agentes, lo que abre sus puertas y los pone a trabajar.

Permisos por identidad de hilo

En la versión para FreeRTOS, la biblioteca decide si una tarea puede registrar, suspender o eliminar agentes revisando si corre con la misma prioridad que el AMS. La nueva base compara, en cambio, el identificador del hilo que llama con el del AMS, o con el del hilo que creó la plataforma durante el arranque. La razón del cambio es que en Linux las prioridades de tiempo real no están garantizadas. Si el proceso no logra fijarlas, todos los hilos quedan con la misma

prioridad, y una comparación de prioridades autorizaría a cualquier hilo, no solo al AMS. La comparación por identidad no tiene ese problema.

Prioridades

Las funciones de prioridad de FreeRTOS se implementan con la política `SCHED_FIFO` de Linux, mediante la función `pthread_setschedparam`. En Linux, un proceso sin el permiso `CAP_SYS_NICE` no puede usar planificación de tiempo real. En ese caso, la biblioteca muestra una advertencia una sola vez y sigue con la política normal (`SCHED_OTHER`), en lugar de detenerse. Las prioridades de los agentes pasan a ser solo una sugerencia. Esto fue lo que ocurrió en la máquina virtual de Ubuntu usada en las primeras pruebas. Para que las prioridades tengan efecto en el r  ver, el proceso necesita ese permiso.

3.1.4. Implementaci  n

El cuadro 3.3 resume qu   se hizo con cada archivo.

Cuadro 3.3: Tratamiento de cada archivo de la biblioteca.

Archivo	Tratamiento
<code>Agent.c</code> , <code>SysVars.c</code> , <code>CyclicBehaviour.c</code> , <code>OneShotBehaviour.c</code> , <code>USER_DEF_COND.c</code> <code>CMAES.h</code>	Se copiaron sin cambios porque no dependen del sistema operativo. A los dos archivos de comportamientos solo se les agreg�� la llamada a <code>MAES_CheckSuspend()</code> . Se reescribi�� con la nueva correspondencia de tipos y con la correcci��n de dos errores de la interfaz original.
<code>Agent_Platform.c</code>	Se reescribi��: ciclo de vida, arranque, prioridades y permisos.
<code>Agent_Msg.c</code> , <code>Agent_Organization.c</code> <code>MAES_Queue.c</code>	Se reemplazaron las llamadas a FreeRTOS por sus equivalentes <code>MAES_*</code> ; la l��gica no cambi��. Archivo nuevo: el nuevo buz��n, que no tiene equivalente en la versi��n original.

Durante la implementaci  n aparecieron dos errores en la interfaz original. Las funciones `AID()` y `get_mailbox()` estaban declaradas como si devolvieran un puntero, pero su implementaci  n y todas las llamadas usaban un valor directo. En FreeRTOS esto no se notaba porque los tipos de tarea y de cola son punteros. En la nueva base, el identificador de agente es un `pthread_t`, que no es un puntero, as   que el error se habr  a manifestado. Se corrigieron ambas declaraciones.

Al principio no hab  a un entorno de compilaci  n para Linux disponible, as   que todo el c  digo se revis   a mano antes de compilarlo. Esa revisi  n encontr   un error de alcance de variables. En la biblioteca C de GNU, `pthread_cleanup_push` y `pthread_cleanup_pop` abren y cierran un bloque de llaves. Una variable se declaraba entre las dos llamadas y se le  a despu  s de la segunda, fuera de su bloque,

por lo que el código no habría compilado. Se corrigió declarando la variable antes de `pthread_cleanup_push`.

3.1.5. Pruebas

La biblioteca se probó con dos programas de ejemplo de la versión original, adaptados a la nueva base. Primero, en una computadora x86 con Ubuntu y después en una Raspberry Pi 5 con la misma imagen de Linux construida con Yocto que usa el róver.

El primero, *emisor y receptor*, tiene dos agentes que intercambian un mensaje cada segundo a través de la plataforma. Es la prueba más simple posible de que la creación, el registro y el envío y recepción de mensajes funcionan de principio a fin. Se compiló y funcionó sin errores, alternando indefinidamente los mensajes de envío y recepción.

El primer ejemplo no usa la suspensión y la reanudación, el envío a varios receptores ni los comportamientos de una sola ejecución. Para probar esas funciones se adaptó el segundo ejemplo, *piedra, papel o tijera*. Dos agentes jugadores envían una jugada al azar a un agente árbitro en cada ronda. Cuando el árbitro recibe las dos jugadas, suspende a los jugadores por medio del AMS, decide el ganador y los reanuda para la ronda siguiente. Este ejemplo usa las mismas funciones que necesitarán los agentes del subsistema para pausar y reanudar tareas. Al adaptarlo aparecieron los errores del cuadro 3.4.

Cuadro 3.4: Errores encontrados al adaptar el ejemplo de piedra, papel o tijera.

Error	Dónde	Corrección
<code>clear_all_receiver()</code> borraba la lista de receptores pero no reiniciaba su contador. Cada ronda de envío múltiple ocupaba un espacio más, hasta agotar la lista.	Biblioteca (también en las versiones para FreeRTOS y libcbp)	Reiniciar el contador junto con la lista
La jugada se elegía con <code>rand() % 2</code> , así que la tijera nunca salía.	Ejemplo	Cambiar a <code>rand() % 3</code>
Las cadenas se comparaban con <code>==</code> en lugar de <code>strcmp</code> ; funcionaba solo porque el compilador reutilizaba las mismas constantes.	Ejemplo	Usar <code>strcmp</code>
Se pasaba una variable directamente como formato de <code>printf</code> .	Ejemplo	Usar <code>printf("%s", variable)</code>

El error de `clear_all_receiver()` es el más importante para el subsistema. Sin la corrección, cualquier agente que arme una lista de receptores una y otra vez, como lo haría un agente que distribuye datos de sensores, dejaría de funcionar después de suficientes rondas. Con las correcciones, el ejemplo funcionó durante decenas de rondas seguidas: salieron las tres jugadas, los ganadores fueron correctos y la lista de receptores nunca se agotó.

La primera compilación real mostró además una advertencia que la revisión manual no había anticipado: la función `ConstructorUSER_DEF_COND` se usaba sin estar declarada en ningún archivo de encabezado, tampoco en la versión original. El compilador usado la aceptó, pero las versiones recientes de GCC y Clang la rechazan como error. Se agregó su declaración en `CMAES.h`.

Cada etapa de verificación encontró al menos un error que no se conocía: la revisión manual, la primera compilación y los dos ejemplos. Por eso se hicieron todas, en lugar de detenerse cuando el código compiló.

3.1.6. Limitaciones

- **Un agente no puede eliminarse a sí mismo.** Si un agente pide al AMS que lo elimine, el hilo del AMS tendría que esperar la terminación de su propio hilo, lo que en POSIX tiene un comportamiento indefinido. Ninguno de los ejemplos lo hace. Si algún agente futuro lo necesita, habrá que separar ese hilo con `pthread_detach` y usar otra forma de terminarlo.
- **Quedan advertencias de compilación** heredadas del uso de punteros a funciones con `void` y `void*` en la biblioteca original. No causan fallos, por lo que no han sido modificadas por continuar con las otras partes del trabajo.
- **El buzón no puede sobrescribir.** Cuando el buzón está lleno, el emisor espera o, si el tiempo límite es cero, su mensaje nuevo se descarta. No existe una operación que reemplace el mensaje viejo por el nuevo, como `xQueueOverwrite` en FreeRTOS. La sección 3.3 analiza qué consecuencias tiene esto para los datos de los sensores y qué conviene hacer.

3.2. Diseño del estimador

Esta sección aplica las herramientas del capítulo 2 al caso de Olympus. Cada decisión se justifica con una restricción que ya se explicó antes. La figura 3.5 resume el flujo de datos del estimador que se desarrolla en las secciones siguientes. La odometría entra como entrada del modelo de proceso, (2.6) y (3.1), y el giroscopio como medición, (3.7). La discrepancia entre ambos, (3.10) y (3.11), ajusta el ruido de proceso de la predicción, (3.3) a (3.5), y el reposo detectado con los encoders habilita la ZARU, (3.9).

3.2.1. Punto de partida

Olympus es un prototipo de róver autónomo de exploración desarrollado en el SETEC Lab del Instituto Tecnológico de Costa Rica [3]. Tiene seis ruedas, una suspensión *rocker-bogie* y gira por deslizamiento. Sus computadoras siguen el esquema de dos capas descrito en la sección 2.1.7.

Hoy la plataforma tiene una versión del software que calcula la odometría. El controlador de bajo nivel acumula las cuentas de los encoders de cada lado

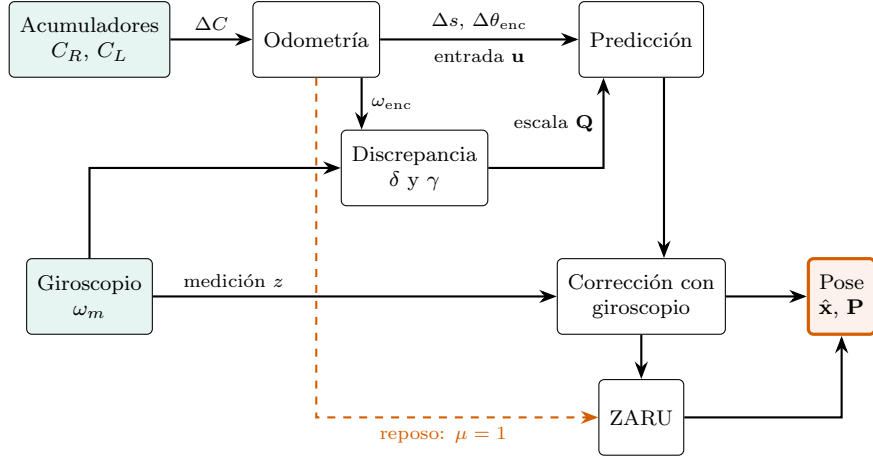


Figura 3.5: Flujo de datos del estimador de pose.

y las envía periódicamente al controlador de alto nivel, que aplica el modelo diferencial (2.8). La documentación de la plataforma propone como trabajo futuro agregar una IMU que reemplace a los encoders para medir los giros, y presenta una primera versión de esa fusión con tres variables de estado [3].

Este trabajo continúa esa propuesta, con tres cambios. Primero, el vector de estado se amplía con la velocidad angular y el sesgo del giroscopio. Segundo, se trata de forma explícita el problema de observabilidad de ese sesgo, con una actualización cuando el ródver está quieto. Tercero, el estimador corre en el controlador de alto nivel, dentro de la arquitectura multiagente, y no en el de bajo nivel.

3.2.2. Odometría de Olympus

El firmware de Olympus cuenta los dos flancos de la señal A de cada encoder y usa la señal B solo para saber el sentido de giro, así que trabaja con decodificación por dos (figura 2.2).

Las seis ruedas se reducen a dos ruedas virtuales. El controlador de bajo nivel suma las cuentas de las tres ruedas de cada lado y envía solo dos acumuladores, uno por lado [3]. En Olympus los tres ejes no tienen el mismo ancho. Las distancias entre centros de rueda son 58,1, 57,1 y 71,4 cm, porque el eje trasero de la suspensión es más abierto. Al sumar las tres ruedas de un lado también se suman sus tres distancias al centro del ródver, y esa suma es tres veces su promedio. En consecuencia, con acumuladores por lado, el ancho de vía nominal que se usa en (2.6) es exactamente el promedio de los tres ejes, $b_{\text{nom}} = 0,622$ m. Cualquier diferencia adicional entre el movimiento real y el de este modelo queda incluida en χ (figura 3.6).

Como el controlador de bajo nivel envía la suma de las tres ruedas de cada

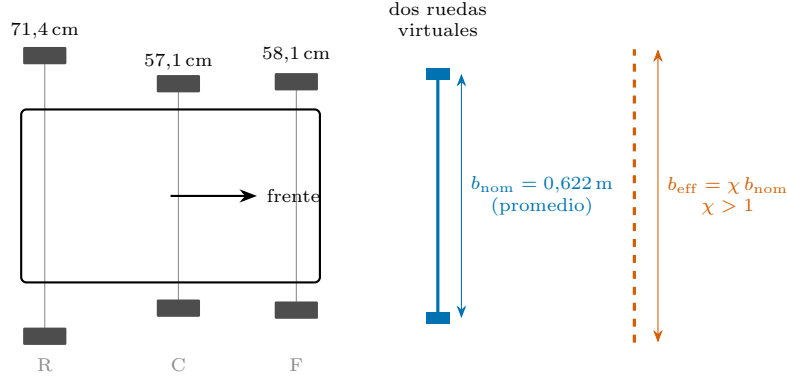


Figura 3.6: Reducción de las seis ruedas de Olympus a un modelo diferencial con ancho de vía efectivo.

lado, el estimador no usa λ_i directamente. Si las tres ruedas de un lado avanzan una distancia d sobre el suelo, la rueda i aporta d/λ_i cuentas, y el acumulador del lado aumenta en $d \sum_i 1/\lambda_i$. La distancia que corresponde a una cuenta del acumulador es entonces

$$\lambda_{\text{lado}} = \left(\sum_{i \in \text{lado}} \frac{N_i}{2\pi r_i} \right)^{-1}. \quad (3.1)$$

Esta constante no es el promedio de las constantes de cada rueda, más bien es cerca de la tercera parte de la constante de una sola rueda. Usar por error el valor de una rueda produciría un error de escala de tres veces, no un ajuste pequeño. Con los valores medidos, la constante del lado derecho y la del izquierdo difieren un 3,7 %.

El controlador de bajo nivel de Olympus envía cuentas acumuladas, que es la segunda opción descrita en la sección 2.1.5. Gracias a ello, el sistema puede funcionar aunque en un intervalo de muestreo se registre menos de una cuenta, sin que el error se acumule. Los acumuladores tienen otra ventaja: si una trama se pierde en el enlace serial, la siguiente trae el total completo, y lo único que cambia es que ese intervalo es más largo. La sección 3.3 muestra que esta protección solo se mantiene si los acumuladores llegan sin convertir hasta el agente que los usa.

3.2.3. Elección del vector de estado

Se usa el siguiente vector de estado:

$$\mathbf{x} = [p_x \ p_y \ \theta \ \omega \ b_\omega]^\top. \quad (3.2)$$

A continuación se explica por qué se incluye cada variable y por qué se dejan otras por fuera.

p_x , p_y y θ forman la pose, que es lo que se quiere estimar. La velocidad angular ω se incluye porque es la única variable que miden los encoders y el giroscopio de forma independiente. Es, por lo tanto, el punto donde la fusión aporta información. El sesgo b_ω se incluye porque, según (2.11), si no se estima, el error del rumbo crece con el tiempo.

Se dejan por fuera las velocidades en el marco de navegación, que sí aparecen en las formulaciones inerciales. La velocidad hacia adelante la da directamente la odometría, con mucho menos ruido del que se obtendría integrando el acelerómetro. Dejar por fuera estas velocidades tiene otra ventaja. La restricción no holonómica (2.4) queda incluida en la forma del modelo y no hace falta la pseudomedida de [23].

También, se dejan por fuera los sesgos del acelerómetro, por el problema de observabilidad de la sección 2.1.5. A la velocidad a la que se mueve Olympus, el acelerómetro no recibe suficiente excitación para que esos sesgos sean observables, y un estado no observable dentro del filtro empeora la estimación (sección 2.1.6).

En Olympus, el acelerómetro mide sobre todo la inclinación (sección 2.1.5). El controlador de bajo nivel lo lee y lo envía, pero el estimador solo lo usa, fuera del filtro, para detectar un vuelco. El reposo se detecta con los encoders (sección 3.2.8), y la corrección por inclinación de la ecuación (2.3) queda fuera del alcance de este trabajo (sección 1.6).

3.2.4. Modelo de proceso

La predicción combina (2.8) con la propagación de las dos variables angulares. Con $\theta_m = \theta + \frac{1}{2} \omega \Delta t$,

$$f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} p_x + \Delta s \cos \theta_m \\ p_y + \Delta s \sin \theta_m \\ \text{wrap}(\theta + \omega \Delta t) \\ \omega_{\text{enc}} \\ b_\omega \end{bmatrix}, \quad \mathbf{u} = \begin{bmatrix} \Delta s \\ \Delta \theta_{\text{enc}} \end{bmatrix}, \quad \omega_{\text{enc}} = \frac{\Delta \theta_{\text{enc}}}{\Delta t}. \quad (3.3)$$

La cuarta fila indica que la velocidad angular no se calcula a partir de su valor anterior, sino que en cada paso se reemplaza por la que da la odometría. Esto significa que la predicción se hace con la odometría. Esta entra como una entrada de control del modelo de proceso y no como una medición. La razón es que el error de la odometría se acumula con el tiempo, y en la formulación bayesiana ese tipo de fuente va en el modelo de proceso. Una medición, en cambio, debe tener un error acotado e independiente de una observación a la siguiente.

El rumbo y la posición se propagan con el ω del estado, es decir, con la velocidad angular del paso anterior después de haber sido corregida por el giroscopio. La odometría de este paso entra en la cuarta fila y el giroscopio la corrige en la etapa de actualización. Así, el rumbo siempre se integra con la mejor estimación de la velocidad angular disponible, a cambio de un retardo de un periodo de muestreo.

El intervalo Δt no lo mide el controlador de alto nivel. Este es la diferencia entre los contadores de tiempo de dos tramas consecutivas del controlador de bajo nivel. En consecuencia, cualquier error en ese contador pasa directamente a ω_{enc} (sección 3.3). Antes de usarlo, el estimador limita Δt al intervalo $[1 \text{ ms}, 500 \text{ ms}]$. Este límite descarta valores imposibles, como los del arranque, los de la vuelta del contador de 32 bits o los de varias tramas perdidas seguidas. No corrige un error de escala del reloj. El modelo de simulación predice que el contador del firmware actual tiene ese error (sección 3.3), pero la predicción todavía no se ha verificado en el r ver. Si se confirma, hay dos formas de corregirlo: estimar la escala en el controlador de alto nivel comparando el contador con un reloj propio, o corregir el reloj en el firmware. La decisi n se tomar  despu s de la verificaci n.

3.2.5. Jacobiano de transici n

Si se deriva (3.3) respecto al estado, con $c_m = \cos \theta_m$ y $s_m = \sin \theta_m$, se obtiene

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & -\Delta s s_m & -\frac{1}{2} \Delta s s_m \Delta t & 0 \\ 0 & 1 & \Delta s c_m & \frac{1}{2} \Delta s c_m \Delta t & 0 \\ 0 & 0 & 1 & \Delta t & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \quad (3.4)$$

La cuarta fila es toda de ceros porque ω se reemplaza en cada paso. Esto tiene un efecto en la predici n (2.15). La incertidumbre anterior de ω no pasa al paso siguiente, y la incertidumbre predicha de ω es exactamente la que indica $\mathbf{Q}$ para ese t rmino, es decir, la incertidumbre de la odometr a.

3.2.6. Ruido de proceso

El error de la odometr a no es igual en todas las direcciones. La incertidumbre del avance Δs est  en la direcci n en que se mueve el r ver. Para representarlo se proyecta con $\mathbf{g} = [c_m \ s_m \ 0 \ 0 \ 0]^\top$:

$$\mathbf{Q} = \mathbf{g} \sigma_{\Delta s}^2 \mathbf{g}^\top + \text{diag}(0, 0, \sigma_\theta^2, \gamma \sigma_{\omega, \text{enc}}^2, \sigma_b^2 \Delta t), \quad (3.5)$$

con

$$\sigma_{\Delta s}^2 = k_\rho |\Delta s| + \frac{\lambda^2}{12}. \quad (3.6)$$

El primer t rmino representa el error que crece con la distancia recorrida y el segundo es el m nimo que impone la cuantizaci n (2.10). El factor $\gamma \geq 1$ es la ganancia adaptativa que se presenta en la secci n 3.2.9.

3.2.7. Modelo de medici n

Seg n (2.11), y si se desprecia el error del factor de escala, la medici n del giroscopio es una funci n lineal del estado:

$$z = \omega + b_\omega, \quad \mathbf{H} = [0 \ 0 \ 0 \ 1 \ \mu], \quad \Sigma_v = \sigma_g^2. \quad (3.7)$$

La sección siguiente explica el indicador $\mu \in \{0, 1\}$. Aquí basta con aclarar su efecto. La medición predicha siempre es $\hat{\omega} + \hat{b}_\omega$, de modo que la innovación $y = z - (\hat{\omega} + \hat{b}_\omega)$ siempre descuenta el sesgo estimado. Lo único que cambia con μ es si la corrección llega también al sesgo. Con $\mu = 0$, la quinta columna de $\mathbf{H}$ es cero; como además la covarianza predicha entre ω y b_ω es cero (cuarta fila de (3.4)), la ganancia sobre b_ω es cero y el sesgo no cambia.

La medición es un solo número, así que $\mathbf{S}$ en (2.16) también es un número y la ganancia (2.17) se calcula con una división, sin invertir matrices. Lo mismo ocurre con la pseudomedida de la sección 3.2.8. En todo el estimador no hay ninguna inversión de matrices, lo que facilita mucho llevarlo a aritmética embebida.

3.2.8. Observabilidad del sesgo y actualización en reposo

Considérese un giro a velocidad angular constante sostenido durante varios segundos. La medición (3.7) solo informa sobre la suma $\omega + b_\omega$, y ninguna otra medición permite separar los dos términos, lo que significa que no son observables por separado. En esa situación, cualquier error sistemático en ω_{enc} (por ejemplo, un ancho de vía efectivo mal calibrado) se puede explicar cambiando b_ω . El filtro termina aceptando la velocidad angular equivocada de la odometría y el aporte del giroscopio desaparece. Este es el caso concreto anunciado en la sección 2.1.6, y la figura 3.7 lo muestra en una simulación con dos giros sostenidos de 60 s y el ancho de vía efectivo sin calibrar. Si el sesgo queda libre durante el movimiento ($\mu = 1$), se aleja del valor verdadero hasta $-0,052 \text{ rad/s}$ y el error de rumbo llega a 408° , casi lo mismo que con la odometría sola. Con el diseño propuesto, el sesgo solo cambia en las pausas (franjas grises) y el error final es de 9° .

Cuando el r  ver est   quieto, el problema desaparece. Si ninguno de los dos acumuladores cambia, la velocidad angular real es cero y, seg  n (2.11), lo que mide el giroscopio es el sesgo. Se define entonces

$$\mu = \begin{cases} 1 & \text{si el r  ver est   quieto,} \\ 0 & \text{si est   en movimiento,} \end{cases} \quad (3.8)$$

y se agrega una pseudomedida de velocidad angular cero, conocida en la literatura inercial como *zero angular rate update* (ZARU) [27]:

$$z_{\text{ZARU}} = 0, \quad \mathbf{H}_{\text{ZARU}} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix}, \quad \Sigma_{\text{ZARU}} = \sigma_{\text{ZARU}}^2 \ll \sigma_g^2. \quad (3.9)$$

Esto cambia la forma de hacer las pruebas. Los momentos en que el r  ver est   quieto dejan de ser pausas sin importancia y pasan a ser parte del m  todo de medici  n, ya que son los   nicos momentos en que se puede observar el sesgo del giroscopio. Entre una pausa y la siguiente, el sesgo estimado no cambia, as   que si el sesgo real se desplaza (por ejemplo, por temperatura, como se vio en la secci  n 2.1.5) el filtro no lo corrige. Por esta raz  n, la frecuencia de las pausas se debe elegir tomando en cuenta esa deriva t  rmica.

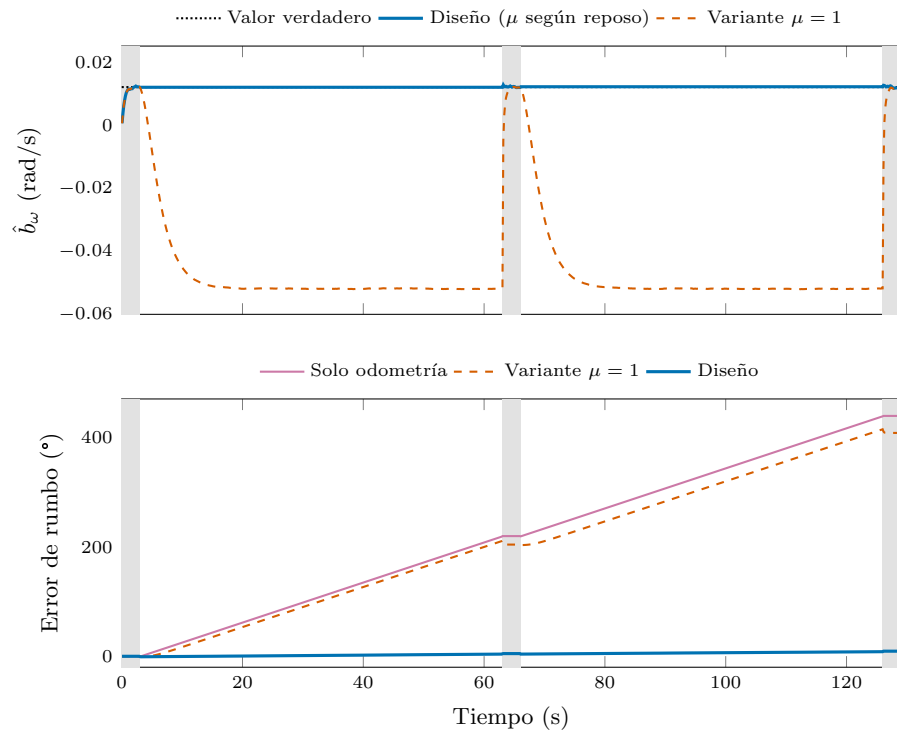


Figura 3.7: Sesgo estimado del giroscopio y error de rumbo simulados en giros sostenidos, con y sin la restricción de estimar el sesgo solo en reposo.

3.2.9. Detección de deslizamiento y adaptación

Las dos estimaciones de la velocidad angular están disponibles antes de hacer la predicción, así que se puede calcular un indicador de discrepancia:

$$\delta = \left| \omega_m - \hat{b}_\omega - \omega_{\text{enc}} \right|. \quad (3.10)$$

Si no hay deslizamiento, δ tiene el tamaño del ruido de los dos sensores juntos. Un valor mucho mayor indica que las ruedas giran sin producir el movimiento que les corresponde. En ese caso se aumenta la incertidumbre de la odometría con el factor

$$\gamma = \begin{cases} 1 & \delta \leq \delta_{\text{th}}, \\ \text{mín} \left(1 + \kappa \left(\frac{\delta}{\delta_{\text{th}}} - 1 \right), \gamma_{\text{máx}} \right) & \delta > \delta_{\text{th}}, \end{cases} \quad (3.11)$$

que multiplica al término $\sigma_{\omega, \text{enc}}^2$ de (3.5).

Es importante aclarar sobre qué matriz actúa esta adaptación. Como la odometría entra por el modelo de proceso (3.3) y no por el de medición, la incertidumbre de los encoders está en $\mathbf{Q}$. Si ante un deslizamiento se aumentara Σ_v , se reduciría la confianza en el giroscopio, que es justamente el sensor que el deslizamiento no afecta. Lo que se busca es que el estimador se apoye en el giroscopio mientras las ruedas patinan, que eso se logra aumentando el ruido de proceso.

El valor de δ_{th} debe elegirse según el ruido de δ a la velocidad real del r  ver. A 2,9 cm/s, un deslizamiento de 30 % en un lado produce una discrepancia de apenas 0,011 rad/s, mientras que sin deslizamiento δ no pasa de 0,004 rad/s. Un umbral de 0,006 rad/s separa ambos casos (figura 3.8). En la figura, el pico al inicio ocurre porque el sesgo del giroscopio todav  a no se ha estimado, y desaparece en la primera pausa. El valor provisional del modelo, 0,10 rad/s, nunca se alcanzar  a.

El indicador (3.10) solo detecta el deslizamiento cuando produce un error de giro. Si las seis ruedas patinan por igual y el r  ver no avanza, la odomet  a y el giroscopio coinciden en que no hay giro, y el estimador suma un desplazamiento que no ocurri  . Con los sensores disponibles, esta situaci  n no se puede detectar.

La detecci  n de fallos de giroscopios por medio de agentes [43] sigue la misma idea que esta detecci  n de deslizamiento. Se compara dos fuentes que deber  an coincidir y usa la diferencia para decidir en cu  l confiar.

3.2.10. Correspondencia con la implementaci  n

El modelo de referencia (secci  n 3.3) implementa directamente las ecuaciones anteriores en cuatro archivos de MATLAB. Estos cuatro archivos son los   nicos que se convertir  n en c  digo C para el controlador de alto nivel. El cuadro 3.5 muestra qu   ecuaciones contiene cada uno.

La divisi  n entre `sep_odometry` y `sep_ekf_step` se da porque el primer archivo contiene todo lo que depende de la geomet  a del r  ver, mientras que el

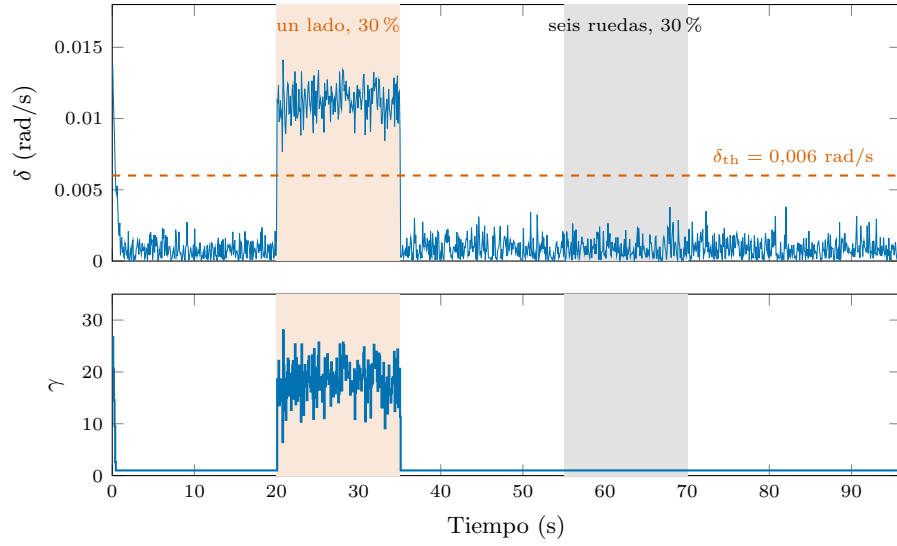


Figura 3.8: Discrepancia δ y factor de adaptación γ simulados en una recta con deslizamiento de un lado y de las seis ruedas.

Cuadro 3.5: Correspondencia entre las ecuaciones y el modelo de referencia.

Ecuación	Archivo	Contenido
(3.1), (2.7)	<code>sep_geo_params</code>	Parámetros geométricos y constante por lado
(2.6), (2.8)	<code>sep_odometry</code>	Conversión de los dos acumuladores en Δs , $\Delta \theta$ y detección de reposo
(3.3), (3.4), (3.5)	<code>sep_ekf_step</code>	Predicción
(3.7), (2.18)	<code>sep_ekf_step</code>	Corrección con el giroscopio
(3.9)	<code>sep_ekf_step</code>	Actualización en reposo
(3.10), (3.11)	<code>sep_ekf_step</code>	Detección de deslizamiento
(3.6), δ_{th} , κ	<code>sep_ekf_params</code>	Parámetros de sintonización

segundo trabaja solo con magnitudes físicas. Esta separación reproduce en la simulación la división entre el agente de fusión de datos y el agente de estimación de la arquitectura final.

3.2.11. Resumen de decisiones de diseño

El cuadro 3.6 resume las decisiones tomadas en esta sección. Para cada una indica qué otras opciones se evaluaron y por qué se eligió la que se usa, siguiendo el formato de los registros de decisiones de arquitectura.

Cuadro 3.6: Decisiones de diseño del estimador y su justificación.

Decisión	Opciones evaluadas	Justificación
Odometría en la predicción y giroscopio en la medición	Predicción con la IMU y corrección con los encoders	El error de la odometría se acumula, y ese tipo de fuente va en el modelo de proceso, no en el de medición. En simulación, la opción contraria fue además mucho más sensible a errores en el ancho de vía.
Vector de estado de cinco variables	Tres variables; formulación inercial de ocho variables	La velocidad angular es la variable que miden dos sensores y el sesgo controla la deriva del rumbo. Las variables inerciales extra son poco observables a la velocidad de Olympus.
Estimar el sesgo solo en reposo	Estimar el sesgo todo el tiempo	Durante un giro sostenido, la velocidad angular y el sesgo no se pueden separar. Si el sesgo queda libre, absorbe el error de calibración y anula el aporte del giroscopio.
Adaptar el ruido de proceso ante deslizamiento	Adaptar el ruido de medición	Con la odometría en la predicción, la incertidumbre de los encoders está en Q . Aumentar la covarianza de medición castigaría al sensor que el deslizamiento no afecta.
Dejar el GNSS fuera del filtro	Usarlo como medición de posición	Su error es unas diez veces mayor que el error permitido al estimador en las trayectorias de prueba, y no es independiente entre muestras.

3.3. Modelo de simulación de referencia

El modelo de simulación es la herramienta con la que se diseñó el filtro, se estimó cuánto error aporta cada fuente y con la que se validará la implementación final contra una verdad de terreno conocida. Está escrito en MATLAB y Simulink.

Antes de describir el modelo, es necesario precisar qué puede demostrar. Varios parámetros del rover todavía no se han medido y los tiempos de las tareas del controlador de bajo nivel son estimaciones. En consecuencia, el modelo sirve para mostrar que el algoritmo funciona y para predecir el comportamiento temporal de la plataforma, pero todavía no demuestra que el rover vaya a cumplir su meta de precisión.

3.3.1. De la primera versión a la versión 2.0

La primera versión del modelo simulaba solo el filtro. Los datos de los sensores aparecían con el mismo reloj que la planta, como si el controlador de bajo nivel midiera el tiempo de forma perfecta. Esa simplificación ocultó la fuente de error más importante de la plataforma, que resultó estar en la forma en que el controlador de bajo nivel mide el tiempo y no en el filtro.

La versión 2.0 simula los dos controladores y el enlace entre ellos. Los dos relojes, el real y el que cree tener el controlador de bajo nivel, se llevan por separado.

3.3.2. Arquitectura

El modelo es una cadena de cinco etapas (figura 3.9). Cada etapa es una función que avanza un paso base de 0,5 ms: `plant_step`, `llc_step`, `channel_step` y `hlc_step`. Un ciclo del controlador de bajo nivel que dura más de lo previsto es simplemente una cantidad mayor de pasos base, y por eso el modelo puede representar un ciclo cuya duración real no coincide con la que reporta.

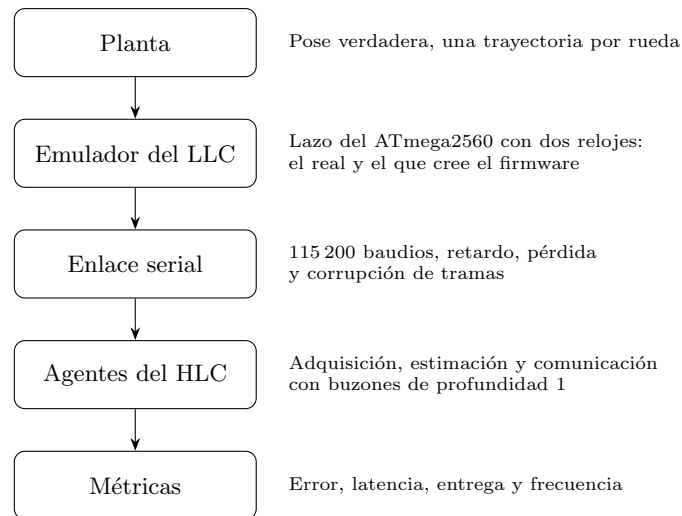


Figura 3.9: Cadena de bloques del modelo de simulación 2.0.

El modelo se puede ejecutar de dos formas: como modelo de Simulink o como

un ciclo de MATLAB. En ambos casos se llaman las mismas funciones, así que la lógica no está duplicada. El ruido tampoco se genera por separado en cada forma. Las secuencias aleatorias se crean una sola vez y se entregan a las dos como señales. Así, las dos formas deben dar exactamente el mismo resultado y la diferencia máxima entre ambas sirve para comprobar que ejecutan el mismo algoritmo.

Planta

La planta calcula la pose verdadera del r  ver y el arco que recorre cada rueda. Cada una puede patinar con su propia raz  n de deslizamiento, la rueda gira un arco a_i , pero el suelo solo avanza $a_i(1 - s_i)$, y el encoder mide a_i . El deslizamiento se puede aplicar de tres formas: igual en las seis ruedas, distinto en cada lado o en una sola rueda. La primera reproduce el caso que el estimador no puede detectar (secci  n 3.2.9). La planta usa los anchos medidos de cada eje y un ancho de v  a efectivo verdadero, adem  s de un cabeceo y un alabeo oscilantes de $1,5^\circ$ y $1,0^\circ$, que representan el balanceo normal de la suspensi  n rocker-bogie.

Emulador del controlador de bajo nivel

El emulador reproduce el lazo principal del firmware 2.20 del ATmega2560 tal como est   escrito. Cada ciclo lee la IMU y los encoders. Cada cinco ciclos lee el sensor ultras  nico, cada 25 ciclos lee los canales anal  gicos y cada 50 ciclos env  a la telemetr  a extendida. El emulador suma el costo estimado de cada tarea para calcular la duraci  n real del ciclo. Tambi  n reproduce detalles que afectan la estimaci  n, como que la IMU y los encoders se leen en momentos distintos del ciclo, pero viajan en la misma trama como si fueran simult  neos; el giroscopio se cuantiza con su escala real; las cuentas se guardan en acumuladores enteros de 32 bits; y las cuentas de cada lado se suman antes de enviarse.

El emulador tiene dos par  metros que permiten comparar el firmware actual con el propuesto. El modo de reloj puede ser *de software*, como el firmware actual, o *con temporizador*. El modo de transmisi  n puede ser *bloqueante*, como el actual, o *por interrupci  n*.

Enlace serial

El enlace modela el puerto serial del ATmega2560 hacia el USB de la Raspberry Pi 5, a 115 200 baudios. Se modela por trama y no por byte, porque un byte tarda $86,8 \mu s$, menos que el paso base. Cada trama tiene un retardo proporcional a su tama  o, puede perderse con una probabilidad de 0,5 % (el indicador admite hasta 1 %) y puede llegar con un byte alterado. La trama del firmware actual est   en texto ASCII y no tiene c  digo de verificaci  n, as   que un byte alterado produce un n  mero cre  ble que ning  n analizador detecta.

Agentes del controlador de alto nivel

El agente de adquisición se activa cada vez que llega una trama, la interpreta y deja en el buzón los acumuladores, el contador de tiempo y la lectura del giroscopio, sin convertirlos. El agente de estimación se activa cada 20 ms, calcula los incrementos, revisa que sean creíbles, ejecuta `sep_odometry` y `sep_ekf_step`, y entrega la pose al agente de comunicación. La revisión de credibilidad descarta intervalos o saltos de cuentas imposibles, y es la única defensa disponible contra los bytes alterados. El buzón entre los dos agentes tiene profundidad 1 y reproduce el comportamiento de CMAES. Si llega una trama cuando el buzón todavía está lleno, el mensaje nuevo se descarta. El modelo permite cambiar esta política para comparar tres opciones (sección 3.3.6). La latencia se calcula sumando la edad estimada de la muestra, el costo del filtro, el costo de publicar y la variación en el tiempo de activación de los hilos de Linux.

3.3.3. Parámetros y su procedencia

Cada parámetro del modelo tiene una etiqueta que indica de dónde sale, si es medido y verificado en el laboratorio, derivado de otros parámetros, provisional (tomado de una hoja de datos o de una campaña anterior) o pendiente. En cada ejecución, el modelo imprime un informe que lista los parámetros pendientes, compara la frecuencia de muestreo real con los 50 Hz requeridos e indica que los resultados no son concluyentes mientras haya parámetros pendientes.

La campaña de medición del 14 de septiembre de 2026 obtuvo el diámetro efectivo de cada rueda (entre 10,6 y 10,7 cm), las cuentas por vuelta de cada rueda y el ancho de cada eje. Siguen pendientes, entre otros, el ancho de vía efectivo χ , el umbral de deslizamiento δ_{th} , el sesgo del giroscopio y su deriva con la temperatura. La velocidad máxima del r  ver en el suelo, 2,9 cm/s al 100 % del ciclo de trabajo, es provisional.

3.3.4. Verificaci  n del modelo

El modelo incluye tres conjuntos de pruebas. La primera comprueba que todos los par  metros que lee el c  digo existan en las dos rutas de ejecuci  n, ya que en MATLAB un campo que falta solo falla a mitad de una simulaci  n. La segunda tiene diez casos que revisan, entre otras cosas, que la constante por lado sea cerca de la tercera parte de la de una rueda, que el signo de cada lado sea correcto, que el sesgo solo cambie en reposo, que el analizador de tramas rechace datos inv  lidos y que el buz  n lleve acumuladores y no incrementos. La tercera reproduce el vector de prueba del contrato de datos, de modo que se puede comparar con las implementaciones en C y en Python.

Los cuatro archivos del n  cleo del estimador (`sep_geo_params`, `sep_ekf_params`, `sep_odometry` y `sep_ekf_step`) cumplen las reglas del generador de c  digo de MATLAB. Estos no agregan campos a una estructura despu  s de leerla, no usan arreglos de tama  o variable y no reservan memoria de forma din  mica. Estas son las mismas reglas que impondr   la generaci  n

de código C para el controlador de alto nivel. El resto del modelo, incluido el emulador del controlador de bajo nivel, no necesita cumplirlas porque nunca se convertirá en código.

3.3.5. Resultados

El cuadro 3.7 resume los resultados del modelo en las siguientes condiciones: un cuadrado UMBmark de 2 m de lado (8 m de recorrido), un sesgo del giroscopio de 0,012 rad/s, la asimetría medida entre los dos lados, la cuantización de los encoders y una pérdida de tramas de 0,5 %.

Cuadro 3.7: Resultados del modelo 2.0 según la configuración del controlador de bajo nivel. La primera fila corresponde al firmware actual.

Reloj y transmisión	Ciclo	Muestreo	Reloj reportado	Error final	Error de rumbo
Software y bloqueante	30,6 ms	32,7 Hz	65,5 %	45,4 %	102,0°
Software e interrupción	23,3 ms	43,0 Hz	85,9 %	—	—
Temporizador y bloqueante	20,4 ms	49,0 Hz	100 %	—	—
Temporizador e interrupción	20,0 ms	50,0 Hz	100 %	1,1 %	2,2°

Con el firmware actual, el error final es de cerca de 45 % de la distancia recorrida. Con el temporizador y la transmisión por interrupción, baja a 1,1 %, dentro de la meta de 3 %. En esa configuración, el sesgo estimado del giroscopio termina en 0,0122 rad/s, frente a 0,0120 rad/s del valor verdadero. La figura 3.10 muestra las dos trayectorias estimadas. El recorrido verdadero no es un cuadrado porque los giros se comandan por tiempo y el r  ver gira menos de 90   en cada esquina.

La figura 3.11 muestra, en el mismo recorrido UMBmark con el firmware corregido, por qu   conviene fusionar la odometr  a y el giroscopio (secci  n 2.1.5). El error de la odometr  a solo cambia durante los giros (francas naranjas) y se mantiene en las pausas (francas grises), mientras que el del giroscopio sin corregir crece tambi  n en las pausas. A partir del primer giro, la fusi  n queda por debajo de ambos.

3.3.6. Hallazgos

Cada uno de estos resultados sali   de mediciones o de leer el firmware instalado, y cada uno se puede reproducir con el modelo.

El reloj del controlador de bajo nivel

Este hallazgo es una predicci  n del modelo, construida a partir de la lectura del c  digo del firmware; todav  a no se ha medido en el r  ver. El cambio de firmware que activa la IMU a  n no se ha probado, y las pruebas del reloj dependen

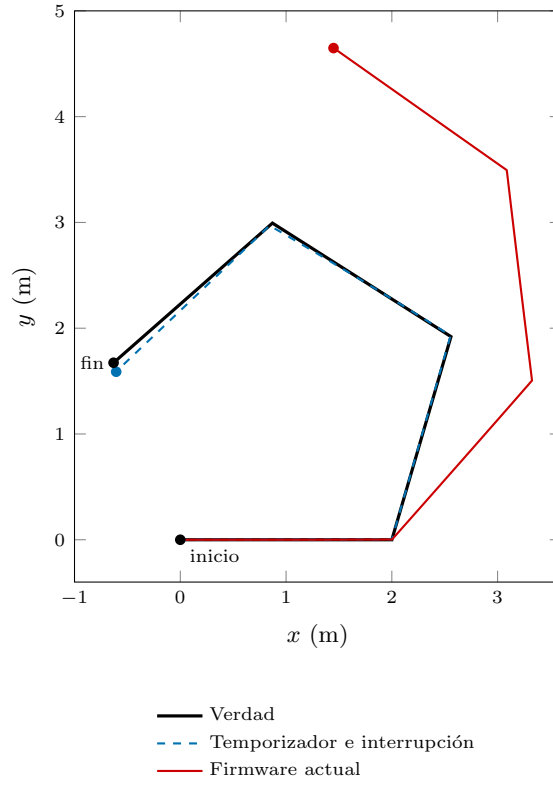


Figura 3.10: Trayectorias simuladas de un recorrido UMBmark de 2m con el firmware actual y con el firmware corregido.

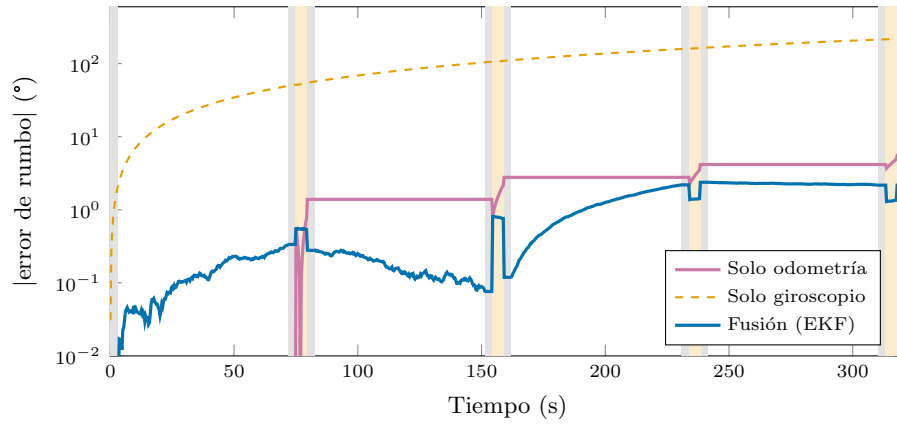


Figura 3.11: Error de rumbo simulado en un recorrido UMBmark de 2m con odometría, giroscopio y fusión.

de ese cambio. Por eso, lo que sigue describe lo que el modelo predice y cómo verificarlo, no una decisión tomada.

Según el código, el firmware cierra cada ciclo sumando 20 ms a su contador de tiempo y esperando 20 ms. No usa ningún temporizador. Una espera al final del ciclo no fija su duración: solo agrega una pausa después del trabajo. El ciclo dura 20 ms más todo el trabajo, pero el contador siempre dice 20 ms (figura 3.12). Según el modelo, el ciclo real duraría cerca de 30 ms y el reloj reportado avanzaría a un 65 % del tiempo real. Si eso se confirma, los 50 Hz del objetivo OE2 no se podrían alcanzar con este firmware, aunque se optimizara todo lo demás.

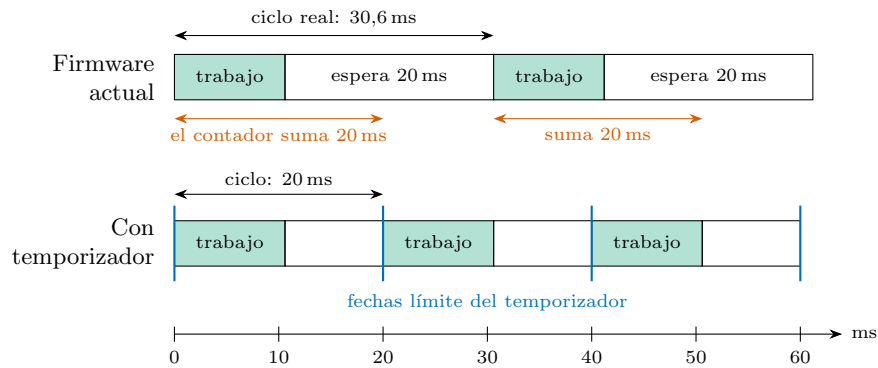


Figura 3.12: Duración del ciclo del controlador de bajo nivel con el firmware actual y con temporizador.

En el modelo, el error aparece al girar. El estimador calcula $\omega_{\text{enc}} = \Delta\theta/\Delta t$ con el Δt reportado, que es menor que el real, así que la odometría dice que el róver gira más rápido de lo que dice el giroscopio. En línea recta, en cambio, los dos modos de reloj dan casi el mismo error (0,11 % y 0,07 %), porque el avance sale de las cuentas y las cuentas no dependen de Δt . Esta es la señal que se debe buscar en los datos reales, un error que crece con el giro acumulado y no con la distancia (figura 3.13).

La corrección tiene dos partes: esperar hasta una fecha límite medida con un temporizador, en lugar de esperar un tiempo fijo, y enviar las tramas por interrupción. El temporizador es el cambio principal porque por sí solo sube el muestreo a 49,0 Hz. Para llegar a los 50 Hz del indicador hace falta también la transmisión por interrupción. El efecto se puede medir en el róver sin instrumentos adicionales. El controlador de alto nivel ya marca cada trama de telemetría con su propio reloj, y la trama incluye el contador del controlador de bajo nivel, así que comparar los dos da el factor de escala del reloj.

Los acumuladores deben llegar sin convertir al agente que los usa

Los acumuladores protegen contra la pérdida de tramas (sección 2.1.5). Pero si el agente de adquisición los convirtiera en incrementos antes de dejarlos en

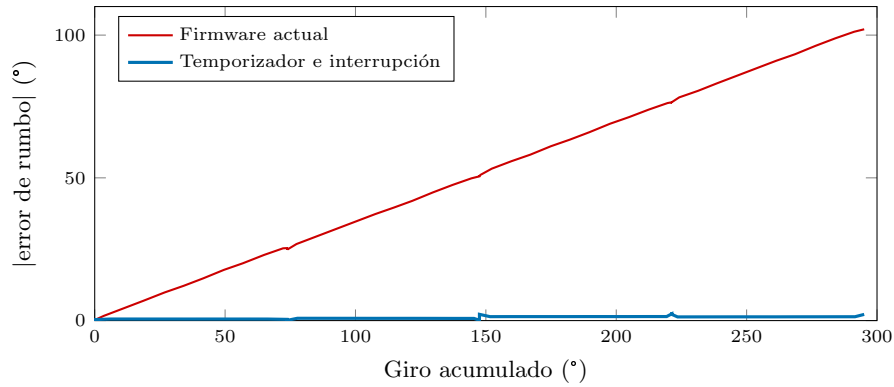


Figura 3.13: Error de rumbo simulado en función del giro acumulado, con el firmware actual y con el firmware corregido.

el buzón, esa protección se perdería un paso más adelante. Cuando un buzón de profundidad 1 está lleno, siempre se pierde el un mensaje; el nuevo, si se descarta, o el viejo, si se sobrescribe. Si ese mensaje llevaba un incremento, el incremento se pierde para siempre y se convierte en un error permanente de posición. Por eso, el buzón lleva los acumuladores tal como llegan, y el agente de estimación calcula la diferencia contra el último valor que él mismo procesó. Así, un mensaje perdido solo cuesta resolución en el tiempo, no distancia, sin importar cuál de los dos se pierda. La regla general es convertir a incrementos en el último paso, nunca antes de un buzón que pueda perder mensajes. Una de las pruebas del modelo verifica esta regla.

La política del buzón

Cuando el agente de adquisición encuentra lleno el buzón del agente de estimación, hay tres cosas que puede hacer. El modelo permite elegir cualquiera de ellas para medir su efecto en lugar de suponerlo:

- **Descartar el mensaje nuevo.** Es lo que hace CMAES cuando se envía con tiempo límite cero. El agente de estimación recibe la muestra más vieja en lugar de la más reciente, lo que agrega a la latencia hasta un periodo del agente de estimación.
- **Sobrescribir el mensaje viejo.** Es lo más conveniente para datos de sensores, donde la muestra reciente vale más que la vieja. CMAES no tiene esta operación.
- **Bloquear al emisor.** Es lo que hace CMAES cuando se envía con un tiempo límite mayor que cero. Mientras el agente de adquisición espera, deja de leer el puerto serial. El búfer del puerto se llena y las tramas se

pierden en el enlace. Este caso no empeora la latencia, sino la tasa de entrega de mensajes, que es otro indicador (OE4, al menos 99 %).

Gracias a la regla de los acumuladores, ninguna de las tres políticas cuesta distancia (figura 3.14). Lo que cambia es la edad de la muestra que llega al filtro y, en el caso del bloqueo, la tasa de entrega.

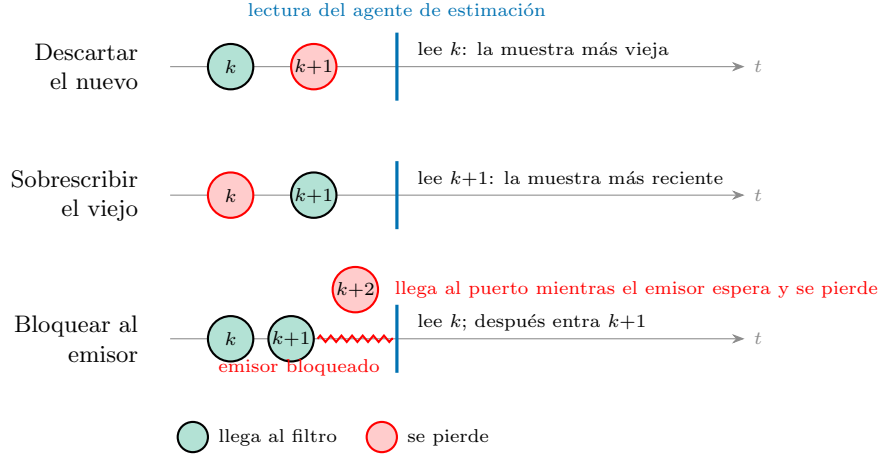


Figura 3.14: Muestra que recibe el filtro con cada política del buzón.

Mientras el agente de estimación sea más rápido que la llegada de tramas, el buzón casi nunca se llena y las tres políticas dan el mismo resultado. La diferencia aparece cuando las dos tasas se igualan. Eso es precisamente lo que ocurre al corregir el reloj del controlador de bajo nivel: con el firmware actual llega una trama cada 30,6 ms, y con el temporizador llega una cada 20 ms, el mismo periodo del agente de estimación. Las dos correcciones, la del reloj y la del buzón, se afectan entre sí y se deben evaluar juntas.

De aquí salen dos recomendaciones. En la ruta de los sensores, el agente de adquisición siempre envía con tiempo límite cero. Así se evita el peor caso, que es el bloqueo y se acepta el costo de latencia de descartar el mensaje nuevo. Por otro lado, la mejora es agregar a la biblioteca una operación de sobrescritura, `MAES.QueueOverwrite`. Esta operación no se puede agregar sola. Como CMAES pasa punteros, sobrescribir el mensaje del buzón no protege el contenido al que apunta, ya que el emisor podría reutilizar la memoria de un mensaje que el receptor todavía está leyendo. Por eso, la sobrescritura solo es segura junto con la rotación de búferes descrita en la sección 3.1.

El acelerómetro no puede detectar el deslizamiento en Olympus

La desaceleración completa del rover al frenar es de unos 0,145 m/s², mientras que un solo grado de cabeceo proyecta 0,171 m/s² de gravedad sobre el eje longitudinal. La suspensión rocker-bogie cambia de cabeceo varios grados todo

el tiempo, así que la señal del frenado queda escondida bajo la inclinación. La figura 3.15 lo muestra con los parámetros del modelo: un frenado de $0,145 \text{ m/s}^2$ y un cabeceo de $1,5^\circ$ a $0,4 \text{ Hz}$. Esta es también la razón cuantitativa para dejar los sesgos del acelerómetro fuera del estado (sección 3.2.3).

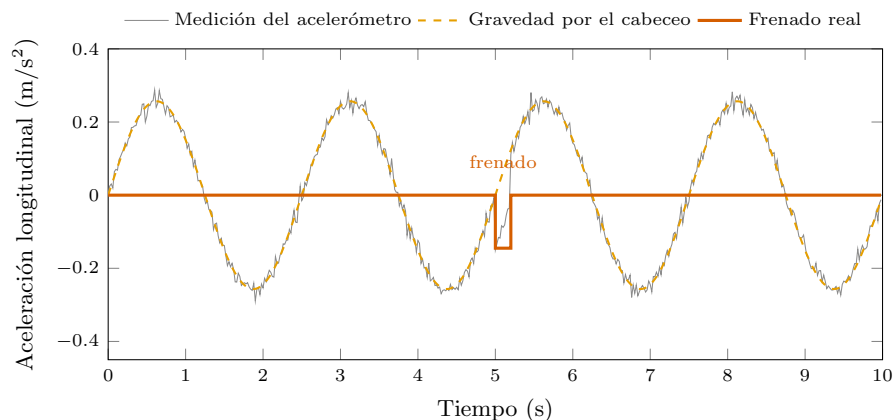


Figura 3.15: Aceleración longitudinal durante un frenado con el cabeceo de la suspensión.

La calibración más importante es la de la escala

El cuadrado UMBmark no mide la escala de distancia (sección 2.1.8) y el giroscopio hace que el filtro sea casi insensible al ancho de vía. En consecuencia, las cuentas por vuelta y el radio de las ruedas importan mucho más que el ancho de vía efectivo. Por ejemplo, la diferencia de $3,7 \%$ entre las constantes de los dos lados produciría, sin calibrar, unos $6,9^\circ$ de desvío en una recta de 2 m . El giroscopio corrige ese desvío, porque el rumbo ya no sale de la diferencia entre lados. Lo que el giroscopio no corrige es un error de escala común a los dos lados: si ambos están mal en $1,8 \%$, la distancia estará mal en $1,8 \%$. La figura 3.16 lo muestra con un barrido en el modelo. Un error de escala de 4% produce un error máximo de posición de 6 a 7 cm , mientras que un error de 20% en el ancho de vía efectivo produce menos de 3 cm .

Anomalía abierta: cuentas por vuelta

La campaña de septiembre midió unas $46\,400$ cuentas por vuelta de rueda, con una diferencia sistemática de $24,8 \%$ entre ruedas (figura 3.17). Hay dos problemas con ese valor. Primero, implicaría una reducción de unas 2111 vueltas del motor por vuelta de rueda con un sensor Hall de 11 pulsos por vuelta y decodificación por dos, y ese motor no tiene esa caja de reducción. Segundo, las cuentas por vuelta dependen del encoder y de la caja, no del diámetro, así que la diferencia entre ruedas no tiene explicación geométrica.

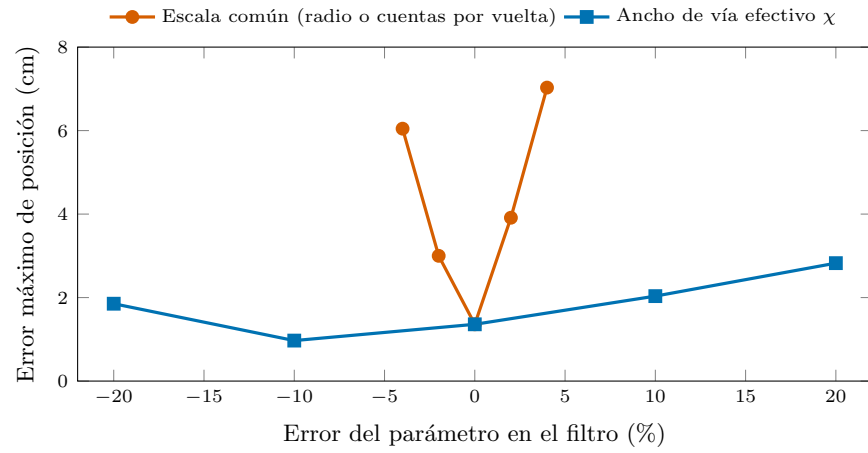


Figura 3.16: Error máximo de posición simulado según el error de calibración de la escala y del ancho de vía efectivo.

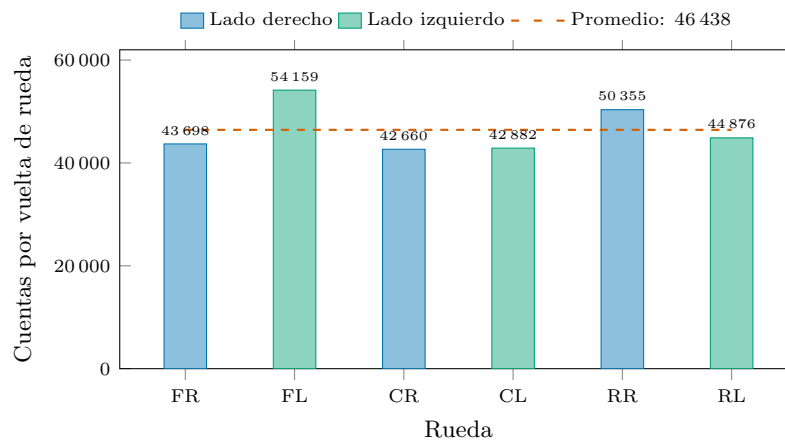


Figura 3.17: Cuentas por vuelta medidas en cada rueda de Olympus.

Una hipótesis explica los dos problemas: rebotes en los flancos del sensor Hall, que agregan cuentas falsas y que dependen de la distancia entre el imán y el sensor, distinta en cada unidad. Se puede comprobar sin volver al banco de pruebas, porque la campaña midió a 20, 50 y 80 % del ciclo de trabajo. Si las cuentas por vuelta aumentan con la velocidad, la causa es el rebote. Si no cambian, la diferencia entre ruedas es real.

3.3.7. Diferencias entre el modelo y la implementación

- **Es un modelo del controlador de bajo nivel, no el controlador.** Los costos de cada tarea son estimaciones. Para confirmarlos hay que medirlos, por ejemplo cambiando el estado de un pin al inicio de cada ciclo y observándolo con un osciloscopio. Cerrar esta diferencia por completo requeriría pruebas con el hardware en el lazo.
- **El tiempo está cuantizado en pasos de 0,5 ms**, lo que representa 1,6 % de un ciclo de 30 ms. Es mucho menos que el error de reloj que se estudia, pero el modelo no puede mostrar efectos más finos.
- **La fusión de datos y la estimación corren en el mismo paso.** En la arquitectura final son dos agentes con un buzón entre ellos. En el modelo, `sep_odometry` y `sep_ekf_step` se ejecutan juntas dentro del agente de estimación, así que el efecto de ese buzón adicional no se simula.
- **No se modelan** la compensación por inclinación, la deriva térmica del sesgo del giroscopio ni el caso en que las seis ruedas patinan por igual, que ya se discutieron en las secciones 1.6 y 3.2.9.

3.3.8. Consecuencias para el proyecto

El modelo cambió cuatro cosas del plan de trabajo. Primero, antes de dar por cumplido el indicador de 50 Hz del OE2, hay que verificar el reloj del controlador de bajo nivel con el firmware que activa la IMU; si la hipótesis del modelo se confirma, también habrá que cambiar la forma en que el firmware mide el tiempo y transmite. Segundo, la calibración debe empezar por la escala de distancia (cuentas por vuelta y radio de rueda), con una prueba en línea recta, antes que por el ancho de vía efectivo. Tercero, las pausas con el r ver quieto son parte del m todo de medici n, porque son el  nico momento en que el sesgo del giroscopio es observable; los escenarios del modelo incluyen pausas de 3 s por esta raz n. Cuarto, en la ruta de los sensores el agente de adquisici n debe enviar siempre con tiempo l mite cero, y la pol tica del buz n se debe evaluar junto con el resultado de la verificaci n del reloj.

3.4. Diseño de los agentes del controlador de alto nivel

Esta sección describe cómo se organiza el software del controlador de alto nivel como un conjunto de agentes CMAES. El diseño parte de los requerimientos del documento DRT-SEP-001 y de la interfaz de sensores ICD-PE-002, actualizados con el modelo de simulación 2.0.

3.4.1. Requerimientos

Los requerimientos que rigen el diseño de los agentes son los siguientes:

- **Composición.** La aplicación tiene al menos cuatro agentes CMAES: adquisición, fusión de datos, estimación y comunicación (PE-RF-006).
- **Mensajería.** Los agentes intercambian mensajes por el bus de CMAES con estructuras binarias de tamaño fijo y un esquema de varios búferes, de modo que ningún mensaje entregado tenga contenido sobrescrito por el productor (PE-RF-007). Los acumuladores de los encoders llegan sin convertir al agente que los usa (PE-RF-008). La tasa de entrega entre agentes debe ser de al menos 99 %, sin más de cuatro pérdidas seguidas (PE-RNF-003).
- **Desempeño.** La pose se publica a 10 Hz o más (PE-RNF-001), con una latencia de extremo a extremo menor que 100 ms (PE-RNF-002), durante al menos 15 minutos sin fallos ni fugas de memoria (PE-RNF-005).
- **Portabilidad.** El código es C99, no reserva memoria de forma dinámica después de la inicialización y no depende de Linux fuera de la capa de portabilidad de CMAES (PE-RNF-007).
- **No interferencia.** El subsistema no modifica el canal de telemetría existente ni el software en producción del rover (PE-RNF-006).

3.4.2. Canal de sensores

El firmware v2.20 del LLC ya emite en cada ciclo una trama de texto con los datos que necesita el estimador, por el mismo enlace USART0 a USB y 115 200 baudios que usa la telemetría existente:

```
RAW:<tick>:<ax>:<ay>:<az>:<gx>:<gy>:<gz>:<encL>:<encR>
```

La trama contiene el contador de tiempo del LLC en milisegundos, las mediciones crudas del acelerómetro y del giroscopio, y los acumuladores de cuentas de cada lado. Su prefijo la distingue de la telemetría, de modo que el software existente no se ve afectado. La longitud es variable, entre unos 40 y 81 bytes, y la trama no tiene código de verificación. Por eso, el agente de estimación descarta las muestras con un Δt fuera de $(0, 0,5]$ s o con saltos de cuenta imposibles.

La especificación completa está en el documento de interfaz ICD-PE-002, que también define una trama binaria de 55 bytes con CRC como formato objetivo si el firmware llega a modificarse.

3.4.3. Descomposición en agentes

La figura 3.18 muestra la arquitectura propuesta. Cada agente corre como un hilo del mismo proceso CMAES y tiene una sola responsabilidad:

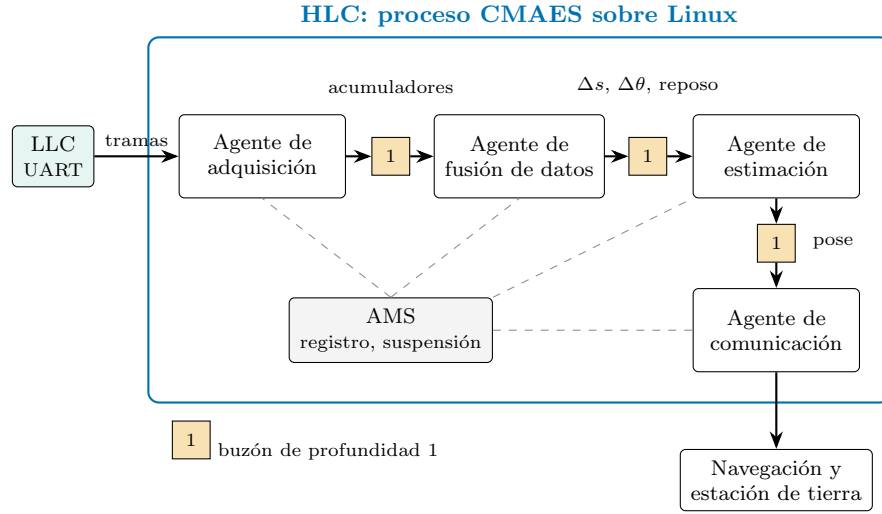


Figura 3.18: Arquitectura de agentes propuesta para el subsistema de estimación de pose.

- **Agente de adquisición.** Lee las tramas del enlace, procesa solo las que empiezan con **RAW**:, las decodifica campo por campo y entrega los acumuladores de los encoders, el contador de tiempo y las mediciones de la IMU sin convertirlos, por la razón explicada en la sección 3.3.
- **Agente de fusión de datos.** Convierte los acumuladores en el avance Δs , el giro $\Delta\theta_{\text{enc}}$ y la velocidad angular ω_{enc} con el modelo diferencial equivalente, y detecta el reposo (PE-RF-009). Corresponde al archivo `sep_odometry` del modelo de referencia.
- **Agente de estimación.** Ejecuta el EKF de la sección 3.2, conserva el estado y su covarianza entre ciclos y detecta el deslizamiento (PE-RF-010 y PE-RF-011). Corresponde al archivo `sep_ekf_step`.
- **Agente de comunicación.** Publica la pose hacia el sistema de navegación existente (PE-RF-012) y registra la pose junto con la referencia del

GPS, con marcas de tiempo del controlador de alto nivel, para el análisis posterior (PE-RF-013).

El AMS de CMAES registra a los cuatro agentes al arrancar y puede suspenderlos o reanudarlos, por ejemplo para detener la estimación durante una calibración.

3.4.4. Mensajes entre agentes

Los mensajes son estructuras binarias de tamaño fijo. Como el buzón de CMAES copia un puntero al contenido y no el contenido, cada agente productor usa una rotación de al menos tres búferes (figura 3.3). En la ruta de los sensores, los agentes envían con tiempo límite cero para no bloquearse cuando el buzón está lleno, según el análisis de la sección 3.3.6. Gracias a que los mensajes llevan cuentas acumuladas, un mensaje descartado no pierde distancia.

3.4.5. Presupuesto de latencia

El cuadro 3.8 reparte el límite de 100 ms entre las etapas del subsistema. El presupuesto suma como máximo 82 ms y deja un margen de 18 ms. Mientras no se verifique el reloj del LLC, la latencia se mide con las marcas de tiempo del controlador de alto nivel.

Cuadro 3.8: Presupuesto de latencia del subsistema.

Etapas	Presupuesto	Fundamento
Muestreo en el LLC	≤ 20 ms	Periodo nominal del lazo del LLC
Lectura de la IMU y armado de la trama	≤ 5 ms	Requerimiento PE-RNF-008
Transmisión de la trama	$\leq 7,0$ ms	Hasta 81 bytes a 115 200 baudios
Recepción y validación (agente de adquisición)	≤ 10 ms	Incluye la espera de planificación
Edad adicional por buzón lleno	≤ 20 ms	Una trama descartada se reemplaza por la siguiente
Agente de fusión de datos	≤ 5 ms	Conversión de acumuladores
Agente de estimación	≤ 10 ms	Costo modelado de unos 3 ms
Agente de comunicación	≤ 5 ms	Costo modelado de unos 2 ms
Total	≤ 82 ms	Margen de 18 ms

El riesgo no está en el cálculo del filtro, sino en la variación del tiempo que tarda Linux en activar cada hilo, que es lo que puede superar el límite en los casos peores. Para reducirla, los agentes usan la política `SCHED_FIFO` con prioridades escalonadas y se fijan a núcleos específicos del procesador. Como se explicó en la sección 3.1, esas prioridades solo tienen efecto si el proceso tiene el permiso `CAP_SYS_NICE`.

3.5. Diseño preliminar de la interfaz eléctrica de la IMU

Esta sección desarrolla el diseño de la interfaz eléctrica del OE1 con la estructura de una decisión de ingeniería: requerimientos, restricciones, alternativas, decisión y entregables. Por ahora es una propuesta basada en las hojas de datos; el montaje y la caracterización medida están planificados para la semana 6 del cronograma.

3.5.1. Requerimientos

- **R1, frecuencia.** El LLC debe leer la IMU dentro de su ciclo de 20 ms. El firmware usa un bus I²C por software a unos 100 kHz, con el que una ráfaga de 14 bytes tarda unos 2 ms, alrededor del 10 % del ciclo.
- **R2, integridad.** Ningún terminal puede recibir una tensión mayor que su máximo absoluto, ni siquiera ante un error del firmware.
- **R3, niveles lógicos.** Las señales deben respetar los márgenes de ruido de la especificación I²C en ambos extremos: 0,1 V_{DD} en nivel bajo y 0,2 V_{DD} en nivel alto [51].
- **R4, tiempo de subida.** El tiempo de subida del bus I²C debe ser menor que 1000 ns, el máximo del modo estándar [51].
- **R5, potencia.** El consumo de la IMU debe caber, con margen amplio, en la corriente disponible de la fuente que la alimenta.

3.5.2. Restricciones

Niveles lógicos. El ATmega2560 funciona a 5 V. En sus pines de uso general, que son los que usa el bus I²C por software, reconoce un nivel alto a partir de 0,6 $V_{CC} = 3,0$ V; en los pines del módulo TWI de hardware el umbral sube a 0,7 $V_{CC} = 3,5$ V [52]. La IMU MPU-9250 funciona a 3,3 V: reconoce un nivel alto a partir de 0,7 $V_{DDIO} \approx 2,3$ V, y sus entradas no admiten más de $V_{DDIO} + 0,5 = 3,8$ V [53], así que no pueden recibir 5 V.

Resistencias de pull-up y capacitancia del bus. En I²C, las líneas solo se llevan a nivel bajo; el nivel alto lo fijan resistencias de pull-up. Su valor mínimo lo limita la corriente que los dispositivos pueden absorber (3 mA en modo estándar), y su valor máximo, el tiempo de subida [51]:

$$R_{p,\text{mín}} = \frac{V_{DD} - V_{OL,\text{máx}}}{I_{OL}}, \quad R_{p,\text{máx}} = \frac{t_r}{0,8473 C_b}. \quad (3.12)$$

Con $V_{OL,\text{máx}} = 0,4$ V, $R_{p,\text{mín}}$ vale 1,53 k Ω del lado de 5 V y 0,97 k Ω del lado de 3,3 V. Para la capacitancia del bus se estima $C_b \leq 50$ pF: unos 20 pF de cable en 20 cm y unos 10 pF por cada terminal conectado. Con $t_r = 1000$ ns, $R_{p,\text{máx}} \approx 23,6$ k Ω .

Potencia. Según Gómez Quesada, el LLC se alimenta por USB desde la Raspberry Pi 5, y la Raspberry Pi 5 tiene un regulador dedicado de 5 V y 3 A [3]. La IMU consume unos 3,5 mA con sus nueve ejes activos [53]. El receptor GPS consume del orden de decenas de miliamperios. Ambos consumos son muy pequeños frente a la capacidad de sus fuentes, así que R5 se cumple con amplitud; el valor real se registra en la caracterización.

Receptor GPS. La conexión del GPS no forma parte del OE1, pero también debe revisarse antes de conectar. Los pines de la Raspberry Pi 5 funcionan a 3,3 V y no toleran 5 V. Si el módulo GPS entrega su salida TX a 3,3 V, se puede conectar directamente a la UART de la Raspberry Pi 5; si la entrega a 5 V, necesita un divisor resistivo o un traductor de niveles. Hay que medirlo antes de conectar.

3.5.3. Alternativas

El cuadro 3.9 compara cuatro formas de conectar la IMU al LLC.

Cuadro 3.9: Alternativas para la interfaz eléctrica entre la IMU y el LLC.

Alternativa	Ventajas	Desventajas
Conexión directa con pull-up a 3,3 V	Sin componentes adicionales	El LLC ve 3,3 V frente a un umbral de 3,0 V: margen de 0,3 V, menor que el de 1,0 V que exige la especificación. Un error del firmware que lleve la línea a 5 V puede dañar la IMU.
Traductor bidireccional con MOSFET (BSS138) [54]	Margen amplio en ambos lados; protege la IMU ante errores del firmware; módulo comercial barato; sin cambios de firmware	Agrega un componente y resistencias en los dos lados.
Traductor integrado para I ² C (por ejemplo, PCA9306)	Margen amplio y especificación formal	Encapsulado de montaje superficial; requiere una placa o un módulo menos común.
Cambiar a SPI con traductores unidireccionales	Más velocidad	Cuatro líneas que traducir; reescribir el controlador de la IMU en el firmware

3.5.4. Decisión

Se elige el traductor bidireccional con MOSFET, con resistencias de pull-up de 4,7 k Ω en ambos lados. Es la única alternativa que cumple R2 y R3 sin cambios de firmware ni componentes difíciles de conseguir. El valor de 4,7 k Ω cae dentro de los límites de (3.12) en ambos lados y da un tiempo de subida

estimado de $0,8473 \times 4,7 \text{ k}\Omega \times 50 \text{ pF} \approx 200 \text{ ns}$, cinco veces por debajo del máximo. Si el módulo de la IMU ya trae resistencias de pull-up, por ejemplo de $10 \text{ k}\Omega$, el valor equivalente en paralelo con las de $4,7 \text{ k}\Omega$ es de unos $3,2 \text{ k}\Omega$, todavía por encima del mínimo. Con el traductor, el LLC ve niveles de 5 V , unos 2 V por encima de su umbral de entrada, y la IMU ve niveles de $3,3 \text{ V}$, 1 V por encima del suyo. Ambos márgenes superan los que exige la especificación I²C: $1,0 \text{ V}$ en el lado de 5 V y $0,66 \text{ V}$ en el lado de $3,3 \text{ V}$.

La IMU se alimenta desde el riel de 5 V del LLC a través del regulador de $3,3 \text{ V}$ del módulo, y el lado de baja tensión del traductor toma su referencia del pin de $3,3 \text{ V}$ del Arduino Mega.

3.5.5. Entregables de hardware

- **Esquemático** de la IMU, el traductor de niveles, las resistencias de pull-up y la conexión del GPS (semana 5).
- **Diagrama de conexión** con la ubicación física de los módulos en Olympus, los pines usados en el Arduino Mega y la Raspberry Pi 5, y la longitud de los cables (semana 5).
- **Informe de caracterización medida** (semana 6), con las tensiones alta y baja y el tiempo de subida en ambos lados del bus, medidos con osciloscopio; el resultado de 45 000 lecturas de la IMU a 50 Hz ; el nivel de la salida del GPS; y el consumo de la IMU y del GPS.

Capítulo 4

Estado de avance del proyecto

Este capítulo resume el estado del proyecto a la fecha de este informe. La sección 4.1 lista las actividades realizadas, y las secciones siguientes interpretan los resultados, describen los retos encontrados y plantean los siguientes pasos.

4.1. Actividades realizadas

Hasta la fecha se realizaron las siguientes actividades:

- **Revisión de literatura y marco teórico.** Se estudiaron la cinemática de vehículos que giran por deslizamiento, los errores de los sensores, la estimación de estado y las arquitecturas multiagente para sistemas embebidos (capítulo 2).
- **Definición de requerimientos.** Se documentaron los requerimientos técnicos del subsistema y el contrato de datos entre el LLC y el HLC.
- **Diseño del estimador.** Se derivó un filtro de Kalman extendido de cinco estados, con estimación del sesgo del giroscopio en reposo y detección de deslizamiento (sección 3.2).
- **Adaptación de CMAES a Linux.** Se escribió una base nueva de la biblioteca sobre hilos POSIX y se validó en Ubuntu con dos programas de ejemplo (sección 3.1).
- **Modelo de simulación.** Se construyó una primera versión que simulaba solo el filtro y luego la versión 2.0, que simula los dos controladores y el enlace serial (sección 3.3).
- **Análisis del firmware del LLC.** La lectura del firmware instalado, junto con el modelo de simulación, llevó a la hipótesis de que el controlador de

bajo nivel no mide el tiempo con un reloj real. La hipótesis todavía no se ha verificado en el r  ver.

- **Campa  a de medici  n.** Se midieron el di  metro efectivo de cada rueda, las cuentas por vuelta, la distancia recorrida por vuelta y el ancho de cada eje.
- **Documentaci  n.** Se redact   este informe con sus figuras y los programas que permiten regenerarlas a partir del modelo.

4.2. Reflexiones preliminares

Los resultados obtenidos hasta ahora son de dise  o y de simulaci  n. Muestran que el enfoque elegido puede cumplir la meta. Con el firmware corregido, el modelo predice un error final de 1,1 % de la distancia recorrida en un recorrido UMBmark, por debajo del 3 % que pide el indicador. Tambi  n muestran un riesgo que no estaba en el plan original, si se confirma la hip  tesis sobre el reloj del controlador de bajo nivel, con el firmware actual el mismo modelo predice un error de 45 %, sin importar lo bien que se ajuste el filtro. La verificaci  n est   pendiente, porque depende de probar el firmware con la IMU activa.

La adaptaci  n de CMAES a Linux est   completa, y los ejemplos funcionan en una Raspberry Pi 5 con el mismo sistema operativo del r  ver; para cumplir el indicador del OE3 faltan la comparaci  n de advertencias con la versi  n original y la prueba de 45 000 mensajes. Con eso queda lista la base para los agentes del OE4. El modelo de simulaci  n, por su parte, ya cumple el papel de referencia que pide el OE6, aunque sus resultados no ser  n concluyentes mientras haya par  metros sin medir. El dise  o de la interfaz el  ctrica del OE1 tiene una propuesta basada en las hojas de datos (secci  n 3.5), pero todav  a falta medirla.

El cuadro 4.1 resume el avance de cada objetivo espec  fico respecto al planeamiento original.

El cuadro 4.2 compara los entregables del cuadro 1.1 con su estado actual. Seg  n el cronograma, a esta fecha deber  an estar cumplidos los hitos H1 a H3. El hito H3 est   cumplido en parte: la biblioteca y sus ejemplos funcionan en la Raspberry Pi 5 con la imagen de Yocto del r  ver, pero faltan la comparaci  n de advertencias y la prueba de 45 000 mensajes. Como la adaptaci  n est   en la ruta cr  tica, el tiempo que tome completarla se descuenta del margen de contingencia. El hito H2 tampoco est   completo, porque el esquem  tico y el diagrama de conexi  n del OE1 no exist  an en el plan original.

En conjunto, el proyecto lleva un avance de [%] respecto al planeamiento original.

4.3. Retos del proyecto

Durante el desarrollo aparecieron los siguientes retos:

Cuadro 4.1: Avance de los objetivos específicos.

Objetivo	Logrado	Pendiente	Avance
OE1	Propuesta de interfaz basada en hojas de datos (niveles, resistencias de pull-up, consumo)	Esquemático, conexión y caracterización medida	[50 %]
OE2	Lectura de encoders por lado; cambio de firmware para activar la IMU; hipótesis sobre el reloj del LLC	Prueba del firmware con la IMU; verificación del reloj; lectura de la IMU a 50 Hz; integración del GPS	[30 %]
OE3	Base nueva sobre hilos POSIX; ejemplos probados en Ubuntu y en la Raspberry Pi 5 con Yocto; errores de la biblioteca corregidos	Comparación de advertencias con la versión original; prueba de 45 000 mensajes a 50 Hz	[90 %]
OE4	Arquitectura de cuatro agentes propuesta; política de buzón analizada	Diseño final e implementación de los agentes; prueba de entrega	[60 %]
OE5	Presupuesto de latencia modelado	Integración en el röver; prueba continua de 15 min	[10 %]
OE6	Modelo de referencia 2.0 con verificación propia; referencia de verdad de terreno definida	Parámetros pendientes; pruebas físicas con marcadores	[80 %]

Cuadro 4.2: Entregables del proyecto y su estado al 30 de septiembre de 2026 (semana 9 del cronograma).

Entregable	Semana	Estado
Documento de requerimientos técnicos	3	Elaborado (DRT-SEP-001)
Esquemático de la interfaz eléctrica de la IMU	5	Pendiente (propuesta en la sección 3.5)
Diagrama de conexión de los sensores	5	Pendiente
Documento de arquitectura multiagente y protocolo serial	5	Parcial: contrato de datos ICD-LLC-002 v1.1 listo; arquitectura de agentes en diseño
Modelo de referencia en MATLAB y Simulink	6	Completo (versión 2.0)
Informe de caracterización eléctrica medida	6	[por confirmar]
Biblioteca CMAES para Linux con informe de pruebas	8	Parcial: ejemplos probados en la Raspberry Pi 5; faltan la comparación de advertencias y la prueba de 45 000 mensajes
Firmware del LLC y módulo del GPS	8	Cambio para activar la IMU sin probar; verificación del reloj pendiente
Aplicación multiagente en el HLC	10	En diseño
Sistema integrado entre el LLC y el HLC	12	Pendiente
Informe de evaluación de precisión	14	Pendiente
Código fuente documentado y manual de integración	16	En curso
Informe final y presentación de defensa	16	En curso (este informe)

- **El reloj del controlador de bajo nivel.** El modelo predice que el firmware actual no alcanza los 50 Hz del OE2 y que su contador de tiempo atrasa (sección 3.3). La predicción no se ha verificado, porque el cambio de firmware que activa la IMU todavía no se ha probado. Si se confirma, habrá que cambiar el firmware, un trabajo que no estaba en el plan original y que afectaría a todas las pruebas físicas.
- **La interfaz eléctrica entró en la ruta crítica.** El OE1 se agregó después del anteproyecto, por observación de los profesores asesores. Su diseño estaba previsto para las semanas 4 y 5, con dos semanas de holgura, y a la semana 9 sigue pendiente. Para reducir el atraso, la parte del firmware que no depende de la interfaz (el reloj con temporizador y la transmisión por interrupción) empieza de inmediato, en paralelo con el diseño eléctrico, y solo la lectura de la IMU espera a la caracterización.
- **Parámetros sin medir.** El ancho de vía efectivo, el umbral de deslizamiento, el sesgo del giroscopio y su deriva con la temperatura siguen pendientes. Mientras tanto, los resultados del modelo no son concluyentes. Las simulaciones mostraron además que el umbral provisional de deslizamiento es unas diez veces mayor de lo que permite la velocidad del röver (sección 3.2.9).
- **La anomalía de las cuentas por vuelta.** Las cuentas medidas no coinciden con la caja de reducción del motor y varían 24,8 % entre ruedas. Hasta resolverla, la escala de la odometría, que es la calibración más importante, tiene una incertidumbre alta.
- **El buzón de CMAES.** La biblioteca pasa punteros y no tiene una operación de sobrescritura, lo que obliga a usar tiempo límite cero en la ruta de sensores y una rotación de al menos tres búferes (secciones 3.1 y 3.3.6).
- **La referencia de verdad de terreno.** A 2,9 cm/s, las trayectorias que el GPS necesita para certificar un error de 3 % tomarían horas. Siguiendo la observación de los profesores asesores, la referencia principal pasó a ser un conjunto de marcadores medidos con cinta métrica, y el GPS quedó como referencia complementaria.
- **El diseño de los agentes sigue abierto.** El modelo todavía ejecuta la fusión de datos y la estimación en el mismo paso, así que el buzón entre esos dos agentes aún no se ha evaluado.

4.4. Siguiendo pasos

Para el próximo mes de trabajo se plantean las siguientes actividades, en orden de prioridad:

1. Completar el esquemático y el diagrama de conexión de la interfaz eléctrica, montar el traductor de niveles (si fuera necesario) y medir niveles, tiempos de subida y consumo (OE1).
2. Probar el firmware del LLC con la IMU activa y verificar el reloj comparando su contador con las marcas de tiempo del HLC. Solo si la hipótesis del modelo se confirma, cambiar el firmware a un temporizador y transmisión por interrupción.
3. Completar la lectura de la IMU en el LLC y verificar el envío de encoders e IMU a 50 Hz (indicador del OE2).
4. Compilar CMAES en la Raspberry Pi 5 con la imagen de Yocto, comparar sus advertencias con las de la versión original y correr la prueba de 45 000 mensajes a 50 Hz para cerrar el OE3 (hito H3); preparar después la prueba de tasa de entrega de mensajes del OE4.
5. Cerrar el diseño de los cuatro agentes e implementarlos con envío de tiempo límite cero y rotación de búferes.
6. Medir la escala de la odometría con una prueba en línea recta, el ancho de vía efectivo con un giro de 360° y el sesgo del giroscopio con la varianza de Allan.
7. Revisar los datos de la campaña a 20, 50 y 80 % de PWM para resolver la anomalía de las cuentas por vuelta.
8. Actualizar el modelo con los parámetros medidos, volver a generar las figuras y definir la longitud mínima de las trayectorias exteriores con GNSS.

Bibliografía

- [1] A. Chaves-Jiménez, “SETEC Lab specialization research and development plan,” in *3rd BIRDS International Workshop*, Aug. 2018.
- [2] M. Gomez-Jenkins, J. Calvo-Alvarado, A. Chaves-Jiménez, J. Carvajal-Godínez, E. Martínez, Y. Arias, A. J. Calvo-Obando, V. Jimenez, J. J. Rojas, A. Valverde-Salazar, and J. Ramirez-Molina, “Project Irazú: Space and ground systems engineering of a 1U CubeSat store and forward mission for environmental monitoring,” *Transactions of the Japan Society for Aeronautical and Space Sciences*, vol. 66, no. 6, pp. 217–225, 2023.
- [3] F. A. Gómez Quesada, “Olympus: Prototipo de robot autónomo de exploración espacial,” June 2026.
- [4] J. Carvajal-Godínez, *Agent-based architectures supporting fault-tolerance in small satellites*. PhD thesis, Delft University of Technology, Delft, Países Bajos, 2021.
- [5] C. Chan-Zheng, “MAES: A multi-agent systems framework for embedded systems,” Master’s thesis, Delft University of Technology, Delft, Países Bajos, 2017.
- [6] C. Chan-Zheng and J. Carvajal-Godínez, “A multi-agent system framework for miniaturized satellite,” *Revista Tecnología en Marcha*, vol. 32, no. 1, pp. 54–67, 2019.
- [7] D. A. Rojas Marín, “FreeMAES: Desarrollo de una biblioteca bajo el paradigma multiagente para sistemas embebidos (MAES) compatible con la NanoMind A3200 y el kernel FreeRTOS,” 2021.
- [8] A. d. J. Conejo Boza, “CMAES: Implementación de la biblioteca FreeMAES basada en el paradigma multiagente para sistemas embebidos en la computadora espacial NanoMind A3200 utilizando el sistema operativo en tiempo real FreeRTOS,” 2022.
- [9] A. Mandow, J. L. Martínez, J. Morales, J. L. Blanco, A. García-Cerezo, and J. González, “Experimental kinematics for wheeled skid-steer mobile robots,” in *Proceedings of the IEEE/RSJ International Conference on*

- Intelligent Robots and Systems (IROS)*, (San Diego, CA), pp. 1222–1227, 2007.
- [10] J. Borenstein and L. Feng, “Measurement and correction of systematic odometry errors in mobile robots,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 6, pp. 869–880, 1996.
 - [11] M. Maimone, Y. Cheng, and L. Matthies, “Two years of visual odometry on the Mars Exploration Rovers,” *Journal of Field Robotics*, vol. 24, no. 3, pp. 169–186, 2007.
 - [12] V. Verma, M. W. Maimone, D. M. Gaines, R. Francis, T. A. Estlin, S. R. Kuhn, G. R. Rabideau, S. A. Chien, M. M. McHenry, E. J. Graser, A. L. Rankin, and E. R. Thiel, “Autonomous robotics is driving Perseverance rover’s progress on Mars,” *Science Robotics*, vol. 8, no. 80, 2023.
 - [13] K. S. Ali, C. A. Vanelli, J. J. Biesiadecki, M. W. Maimone, Y. Cheng, A. M. San Martin, and J. W. Alexander, “Attitude and position estimation on the Mars Exploration Rovers,” in *Proceedings of the IEEE International Conference on Systems, Man and Cybernetics*, vol. 1, (Waikoloa, HI), pp. 20–27, 2005.
 - [14] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*. Cambridge, MA: MIT Press, 2 ed., 2011.
 - [15] K. Ogata, *Modern Control Engineering*. Upper Saddle River, NJ: Prentice Hall, 5 ed., 2010.
 - [16] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ: Princeton University Press, 2008.
 - [17] J. Borenstein, H. R. Everett, L. Feng, and D. Wehe, “Mobile robot positioning: Sensors and techniques,” *Journal of Robotic Systems*, vol. 14, no. 4, pp. 231–249, 1997.
 - [18] K. T. McBrayer, P. R. Chai, and E. L. Judd, “Communication delays, disruptions, and blackouts for crewed Mars missions,” in *ASCEND 2022*, American Institute of Aeronautics and Astronautics, 2022. NASA Technical Reports Server, document 20220013418.
 - [19] G. Lentaris, K. Maragos, I. Stratakis, *et al.*, “High-performance embedded computing in space: Evaluation of platforms for vision-based navigation,” *Journal of Aerospace Information Systems*, vol. 15, no. 4, pp. 178–192, 2018.
 - [20] T. D. Barfoot, *State Estimation for Robotics*. Cambridge: Cambridge University Press, 2017.
 - [21] D. H. Titterton and J. L. Weston, *Strapdown Inertial Navigation Technology*. Londres: Institution of Engineering and Technology, 2 ed., 2004.

- [22] S. M. LaValle, *Planning Algorithms*. Cambridge: Cambridge University Press, 2006.
- [23] G. Dissanayake, S. Sukkarieh, E. Nebot, and H. Durrant-Whyte, “The aiding of a low-cost strapdown inertial measurement unit using vehicle model constraints for land vehicle applications,” *IEEE Transactions on Robotics and Automation*, vol. 17, no. 5, pp. 731–747, 2001.
- [24] J. L. Martínez, A. Mandow, J. Morales, S. Pedraza, and A. García-Cerezo, “Approximating kinematics for tracked mobile robots,” *The International Journal of Robotics Research*, vol. 24, no. 10, pp. 867–878, 2005.
- [25] N. El-Sheimy, H. Hou, and X. Niu, “Analysis and modeling of inertial sensors using Allan variance,” *IEEE Transactions on Instrumentation and Measurement*, vol. 57, no. 1, pp. 140–149, 2008.
- [26] J. Borenstein and L. Feng, “Gyrodometry: A new method for combining data from gyros and odometry in mobile robots,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, vol. 1, (Minneapolis, MN), pp. 423–428, 1996.
- [27] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*. Boston, MA: Artech House, 2 ed., 2013.
- [28] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [29] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [30] P. S. Maybeck, *Stochastic Models, Estimation, and Control*, vol. 1. Nueva York: Academic Press, 1979.
- [31] S. J. Julier and J. K. Uhlmann, “Unscented filtering and nonlinear estimation,” *Proceedings of the IEEE*, vol. 92, no. 3, pp. 401–422, 2004.
- [32] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Nueva York: John Wiley & Sons, 2001.
- [33] D. Simon, *Optimal State Estimation: Kalman, H-infinity, and Nonlinear Approaches*. Hoboken, NJ: John Wiley & Sons, 2006.
- [34] M. Wooldridge, *An Introduction to MultiAgent Systems*. Chichester: John Wiley & Sons, 2 ed., 2009.
- [35] Foundation for Intelligent Physical Agents, “FIPA agent management specification,” Especificación SC00023K, FIPA, 2004.

- [36] J. Borenstein and L. Feng, “UMBmark: A method for measuring, comparing, and correcting dead-reckoning errors in mobile robots,” Informe técnico UM-MEAM-94-22, University of Michigan, Ann Arbor, MI, 1994.
- [37] B. Barshan and H. F. Durrant-Whyte, “Inertial navigation systems for mobile robots,” *IEEE Transactions on Robotics and Automation*, vol. 11, no. 3, pp. 328–342, 1995.
- [38] L. Matthies, E. Gat, R. Harrison, B. Wilcox, R. Volpe, and T. Litwin, “Mars microrover navigation: Performance evaluation and enhancement,” *Autonomous Robots*, vol. 2, no. 4, pp. 291–311, 1995.
- [39] K. Yoshida and G. Ishigami, “Steering characteristics of a rigid wheel for exploration on loose soil,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, (Sendai, Japón), pp. 3995–4000, 2004.
- [40] J. Y. Wong, *Theory of Ground Vehicles*. Hoboken, NJ: John Wiley & Sons, 4 ed., 2008.
- [41] Foundation for Intelligent Physical Agents, “FIPA ACL message structure specification,” Especificación SC00061G, FIPA, 2002.
- [42] F. Bellifemine, A. Poggi, and G. Rimassa, “JADE: A FIPA-compliant agent framework,” in *Proceedings of the 4th International Conference on the Practical Applications of Intelligent Agents and Multi-Agent Technology (PAAM)*, (Londres), pp. 97–108, 1999.
- [43] J. Carvajal-Godínez, J. Guo, and E. Gill, “Agent-based algorithm for fault detection and recovery of gyroscope’s drift in small satellite missions,” *Acta Astronautica*, vol. 139, pp. 181–188, 2017.
- [44] A. I. Mourikis and S. I. Roumeliotis, “A multi-state constraint Kalman filter for vision-aided inertial navigation,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, (Roma), pp. 3565–3572, 2007.
- [45] T. Qin, P. Li, and S. Shen, “VINS-Mono: A robust and versatile monocular visual-inertial state estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [46] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, “Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age,” *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016.
- [47] F. Dellaert and M. Kaess, “Factor graphs for robot perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1–2, pp. 1–139, 2017.

- [48] A. Barrau and S. Bonnabel, “The invariant extended Kalman filter as a stable observer,” *IEEE Transactions on Automatic Control*, vol. 62, no. 4, pp. 1797–1812, 2017.
- [49] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, “The EuRoC micro aerial vehicle datasets,” *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.
- [50] P. Furgale, P. Carle, J. Enright, and T. D. Barfoot, “The Devon Island rover navigation dataset,” *The International Journal of Robotics Research*, vol. 31, no. 6, pp. 707–713, 2012.
- [51] NXP Semiconductors, *I²C-bus specification and user manual*, rev. 7.0 ed., 2021.
- [52] Microchip Technology, *ATmega640/V-1280/V-1281/V-2560/V-2561/V Datasheet*, 2020.
- [53] TDK InvenSense, *MPU-9250 Product Specification*, rev. 1.1 ed., 2016.
- [54] NXP Semiconductors, *Level shifting techniques in I²C-bus design*, 2007.